

东北大学资源与土木工程学院

作 业 报 告

课程名称：GNSS 定位新技术与方法

学生专业：测绘工程

学生班级：测绘 2301

学 号：20232444

学生姓名：闻子博

指导教师：贺黎明

作业成绩：

教师评语：

评阅人：

评阅时间：

目录

一、引言.....	1
1.1 开发背景.....	1
1.2 开发意义.....	2
1.3 开发环境与工具.....	2
1.4 开发任务与目标.....	2
1.3.1 硬件环境.....	2
1.3.2 软件环境.....	2
1.4.1 开发任务.....	2
1.4.2 开发目标.....	3
二、需求分析.....	3
2.1 功能需求.....	3
2.1.1 登录模块.....	3
2.1.2 地图基础功能模块.....	3
2.1.3 定位功能模块.....	3
2.1.4 地点搜索功能模块.....	3
2.1.5 路线规划功能模块.....	3
2.1.6 电子围栏功能模块.....	4
2.1.7 轨迹绘制功能模块.....	4
2.1.8 拍照定位功能模块.....	4
2.1.9 坐标转换模块.....	4
2.1.10 定位精度与质量评估模块.....	4
2.1.11 误差分析、组合导航、欺骗检测模块.....	4
2.1.12 CORS/PPP 模拟改正与实时动态模式模块	4
2.2 非功能需求.....	7

2.2.1 性能需求.....	7
2.2.2 兼容性需求.....	8
2.2.3 易用性需求.....	8
2.2.4 安全需求.....	8
2.3 用户场景需求.....	8
2.4 需求确认与可行性分析.....	8
三、系统设计.....	8
3.1 总体架构设计.....	8
3.2 UI 界面设计	9
3.3 功能模块设计.....	10
3.3.1 登录模块设计.....	10
3.3.2 地图基础模块设计.....	10
3.3.3 定位与实时动态模块设计.....	10
3.3.4 地点搜索模块设计.....	11
3.3.5 路线规划模块设计.....	11
3.3.6 电子围栏模块设计.....	11
3.3.7 轨迹绘制模块设计.....	11
3.3.8 拍照定位模块设计.....	11
3.3.9 GNSS 七项增强模块设计	11
3.4 权限设计.....	11
3.5 数据流程设计.....	12
3.5.1 定位与实时刷新数据流程.....	12
3.5.2 路线规划数据流程.....	12
3.5.3 拍照定位与围栏/轨迹数据流程	12
四、代码实现.....	12

4.1 核心配置文件实现.....	12
4.2 登录与主界面代码实现.....	12
4.3 地图、定位与搜索代码实现.....	12
4.4 路线规划、围栏与轨迹实现.....	13
4.5 拍照定位实现.....	13
4.6 GNSS 七项增强实现	13
4.7 实时动态模式实现.....	13
4.8 典型问题调试与修复.....	13
五、系统测试.....	14
5.1 测试目的与环境.....	14
5.2 功能测试.....	14
5.3 性能测试.....	14
5.4 兼容性与安全性测试.....	15
5.5 测试结果总结.....	15
六、优化方向.....	15
6.1 功能优化.....	15
6.2 性能优化.....	15
6.3 UI 与交互优化	15
6.4 GNSS 能力深化方向	15
七、总结与展望.....	16
7.1 开发总结.....	16
7.2 未来展望.....	16
附录 A module.json5 完整代码	17
A.1 module.json5 清单	17
附录 B string.json 完整代码	19

B.1 string.json 清单	19
附录 C index.ets 完整代码.....	20
C.1 index.ets 清单.....	20
附录 D amap.html 完整代码.....	33
D.1 amap.html 清单.....	33
附录 E 关键函数与调用关系说明	52
E.1 index.ets 关键方法说明.....	52
E.2 amap.html 关键函数说明	53
E.3 关键调用链分析	53
附录 F 主要功能界面图集与版位说明	54
F.1 实时首页与状态总览界面	55
F.2 GNSS 总览实时界面	56
F.3 实时坐标转换输入界面	57
F.4 实时坐标转换结果界面	58
F.5 GNSS 实时精度评估界面	59
F.6 GNSS 实时质量评估界面	60
F.7 GNSS 实时误差源分析界面	61
F.8 GNSS 组合导航待开启界面	62
F.9 GNSS 组合导航运行界面	63
F.10 融合点地图显示效果界面	64
F.11 GNSS 欺骗与异常检测界面	65
F.12 CORS 模拟改正实时界面	66
F.13 PPP 平滑改正实时界面.....	67

摘要

随着鸿蒙系统（HarmonyOS）在移动终端领域的持续推广，基于国产操作系统开展导航定位类应用开发，既具有工程实践价值，也具有自主软硬件生态适配的现实意义。本次开发以鸿蒙原生应用为基础，结合 DevEco Studio、AbilityKit、LocationKit、CameraKit 与 WebView 等官方能力，设计并实现了一款集成高德地图的卫星导航与 GNSS 增强 APP。

与基础版导航应用相比，本版本在登录、地图加载、当前位置显示、地点搜索、驾车/骑行/步行路线规划、电子围栏、轨迹绘制、拍照定位、卫星地图切换等功能之上，进一步构建了实时动态模式与 GNSS 七项增强能力，包括坐标转换、定位精度评估、定位质量评估、导航误差源分析、GNSS+惯性组合导航、GNSS 欺骗与异常检测、CORS/PPP 模拟改正。

本文围绕开发背景、需求分析、系统设计、代码实现、系统测试、优化方向和总结展望等内容，系统梳理了实时动态 GNSS 增强卫星导航 APP 的完整开发流程，并在附录中给出完整源码清单。

关键词

鸿蒙系统；卫星导航；GNSS 实时动态；DevEco Studio；高德地图；LocationKit；组合导航；CORS/PPP

一、引言

1.1 开发背景

在移动互联网、物联网与智能终端不断融合的背景下，导航定位应用已经由传统的“地图查看”工具，逐步演进为集路线决策、位置服务、环境感知、轨迹分析与安全预警于一体的综合平台。尤其是在测绘工程、智慧交通、校园导航、户外作业与应急救援等场景中，用户不仅需要静态地图与基础导航功能，更需要对当前位置、轨迹演化、定位质量与异常状态进行持续感知与分析。

与此同时，HarmonyOS 作为国产自主分布式操作系统，在终端流畅性、跨设备协同、能力组件化与安全权限管理方面形成了有别于传统移动系统的技术路径。基于鸿蒙系统开发导航定位类 APP，既能验证鸿蒙生态对地图服务、相机服务、定位服务与 WebView 交互的支持能力，也有助于形成自主操作系统场景化应用开发的实践样板。

本次开发则进一步面向课程“GNSS 定位新技术与方法”的要求，对原有 APP 进行了实时动态与 GNSS 分析向的二次升级，使系统由“能导航”提升为“能实时观测、能评估、能分析、能演示增强定位思路”的综合实验平台。

1.2 开发意义

从课程实践角度看，本项目将 GNSS 定位理论、导航电子地图调用、移动应用开发与工程调试过程统一到了一个可运行的鸿蒙 APP 中，使抽象的坐标系转换、定位误差、DOP 指标、组合导航与改正思想具备了可视化、交互化与动态化的展示界面。

从技术训练角度看，本次开发覆盖了鸿蒙端权限配置、设备真机调试、签名部署、ArkTS 界面开发、WebView 桥接调用、第三方地图 API 集成、异常处理与 UI 交互优化等一整套流程，对于形成完整的软件工程能力与系统联调能力具有重要意义。

从成果产出角度看，本次开发文档既是对源代码的结构化解释，也是对 APP 从基础版升级为实时动态 GNSS 增强版的系统总结，可为后续继续扩展语音导航、离线地图、多设备协同、真实 NTRIP 接入等方向提供可靠基础。

1.3 开发环境与工具

1.4 开发任务与目标

1.3.1 硬件环境

- 开发设备：Windows 11 笔记本电脑。
- 测试设备：支持 HarmonyOS 的华为手机。
- 网络环境：校园 Wi-Fi 与移动网络。

1.3.2 软件环境

环境项	具体配置
操作系统	Windows 11 64 位
开发工具	DevEco Studio
目标系统	HarmonyOS 手机端
前端技术	ArkTS / ETS、HTML、JavaScript、CSS
地图服务	高德地图 Web API 2.0
鸿蒙能力包	AbilityKit、LocationKit、CameraKit、WebView

1.4.1 开发任务

- 实现基础导航功能与 GNSS 增强功能。

- 增加实时动态模式，使指标自动刷新。
- 编写与参考 PDF 篇幅接近的完整开发文档。

1.4.2 开发目标

功能目标：形成“导航 + 观测 + 评估 + 分析 + 改正模拟”的一体化应用。

性能目标：保证地图初始化、实时定位更新和 GNSS 面板刷新流畅。

文档目标：开发文档结构、篇幅与参考 PDF 接近，内容完整、层次清晰。

二、需求分析

2.1 功能需求

2.1.1 登录模块

系统提供账号与密码输入框，采用预设演示账号进行身份校验。该需求来源于当前源码与课程作业目标的结合，既考虑了基础导航的可用性，也兼顾了 GNSS 教学演示的专业性。

2.1.2 地图基础功能模块

WebView 加载高德地图页面，完成地图初始化、工具栏与比例尺加载、标准图与卫星图切换。该需求来源于当前源码与课程作业目标的结合，既考虑了基础导航的可用性，也兼顾了 GNSS 教学演示的专业性。

2.1.3 定位功能模块

申请位置权限，调用 LocationKit 获取 WGS84 坐标并同步到地图。该需求来源于当前源码与课程作业目标的结合，既考虑了基础导航的可用性，也兼顾了 GNSS 教学演示的专业性。

2.1.4 地点搜索功能模块

通过 PlaceSearch 搜索地点并定位到结果点。该需求来源于当前源码与课程作业目标的结合，既考虑了基础导航的可用性，也兼顾了 GNSS 教学演示的专业性。

2.1.5 路线规划功能模块

支持驾车、骑行、步行三种路线规划模式。该需求来源于当前源码与课程作业目标的结合，既考虑了基础导航的可用性，也兼顾了 GNSS 教学演示的专业性。

2.1.6 电子围栏功能模块

以当前位置为中心绘制圆形围栏，支持添加与删除。该需求来源于当前源码与课程作业目标的结合，既考虑了基础导航的可用性，也兼顾了 GNSS 教学演示的专业性。

2.1.7 轨迹绘制功能模块

支持手动添加和实时模式下自动追加轨迹点。该需求来源于当前源码与课程作业目标的结合，既考虑了基础导航的可用性，也兼顾了 GNSS 教学演示的专业性。

2.1.8 拍照定位功能模块

调用 CameraKit 拍照并将照片 URI 与位置绑定。该需求来源于当前源码与课程作业目标的结合，既考虑了基础导航的可用性，也兼顾了 GNSS 教学演示的专业性。

2.1.9 坐标转换模块

支持 WGS84、GCJ02、BD09 三种坐标互转。该需求来源于当前源码与课程作业目标的结合，既考虑了基础导航的可用性，也兼顾了 GNSS 教学演示的专业性。

2.1.10 定位精度与质量评估模块

统计平均散布、RMS、CEP95 及 NMEA 质量指标。该需求来源于当前源码与课程作业目标的结合，既考虑了基础导航的可用性，也兼顾了 GNSS 教学演示的专业性。

2.1.11 误差分析、组合导航、欺骗检测模块

展示误差源、组合平滑结果与风险等级。该需求来源于当前源码与课程作业目标的结合，既考虑了基础导航的可用性，也兼顾了 GNSS 教学演示的专业性。

2.1.12 CORS/PPP 模拟改正与实时动态模式模块

开启实时模式后自动更新样本，并模拟改正效果。该需求来源于当前源码与课程作业目标的结合，既考虑了基础导航的可用性，也兼顾了 GNSS 教学演示的专业性。

图 2-1 实时首页与动态状态界面



图 2-1 实时首页与动态状态界面：展示搜索框、定位当前位置、开启实时模式、拍照定位、卫星图切换以及经纬度与实时状态卡片。

图 2-2 GNSS 实时坐标转换界面

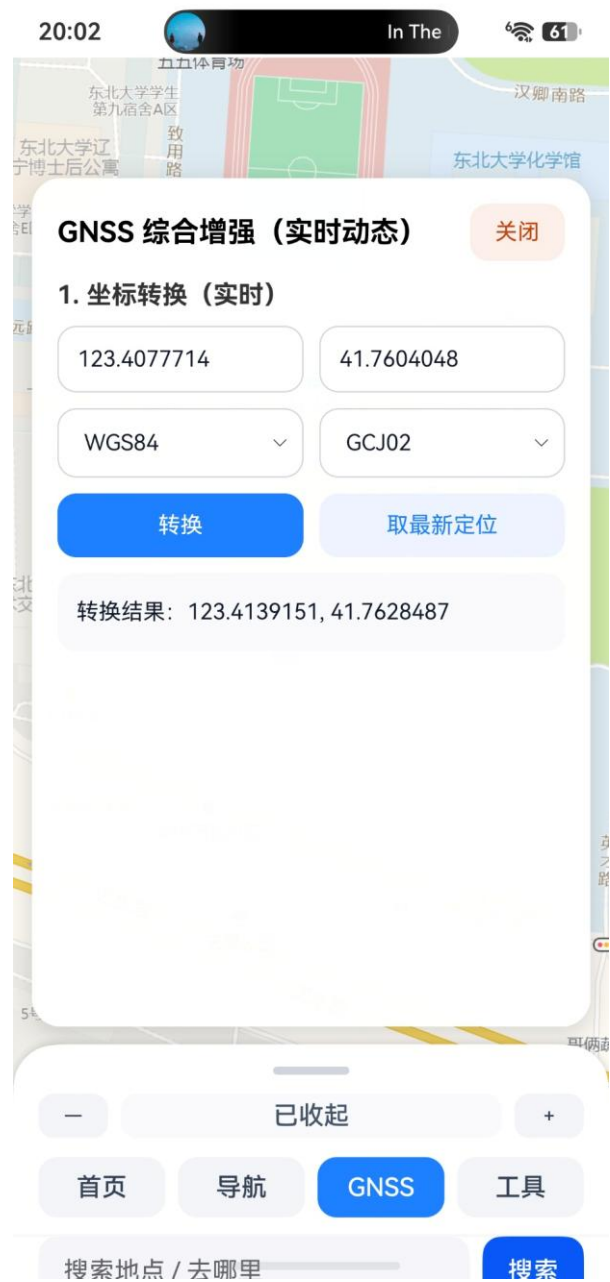


图 2-2 GNSS 实时坐标转换界面：展示实时样本自动填充、WGS84 与 GCJ02 坐标系选择以及实时转换结果。

图 2-3 GNSS 实时精度评估界面



图 2-3 GNSS 实时精度评估界面：展示样本数、平均散布、RMS、CEP95 和均值坐标等核心精度指标。

2.2 非功能需求

2.2.1 性能需求

实时模式下应避免页面冻结与明显卡顿，地图与 GNSS 面板刷新周期应控制在合理范围内。

2.2.2 兼容性需求

系统面向 HarmonyOS 手机设备设计，底部抽屉式布局应适应不同屏幕高度。

2.2.3 易用性需求

用户应能通过首页、导航、GNSS、工具四个页签快速找到对应功能。

2.2.4 安全需求

系统仅申请定位、相机和互联网等必要权限。

2.3 用户场景需求

场景一：日常出行导航。

场景二：GNSS 教学演示。

场景三：外业记录。

场景四：定位质量与异常观察。

2.4 需求确认与可行性分析

从技术可行性来看，鸿蒙端具备定位、相机、WebView 与权限控制所需官方能力，高德地图 Web API 提供了搜索、路线规划与地图交互接口。

从工程可行性来看，项目源码已经形成 `index.ets`、`amap.html`、`module.json5`、`string.json` 四个明确文件边界，结构清晰，便于进行模块化扩展。

三、系统设计

3.1 总体架构设计

系统采用“鸿蒙原生 UI 层 + 业务控制层 + Web 地图与 GNSS 分析层 + 配置资源层”的分层架构。UI 层由 `index.ets` 构成，负责页面搭建、状态维护、按钮交互与功能分栏；业务控制层封装在 `Index` 组件方法中，负责权限申请、定位获取、实时动态轮询、字符串转义与 JavaScript 调用分发；Web 地图与 GNSS 分析层由 `amap.html` 构成，负责地图渲染、路径规划、围栏/轨迹绘制、GNSS 实时面板与指标计算；配置资源层由 `module.json5` 与 `string.json` 组成。

该架构的优点在于，一方面发挥鸿蒙原生端对于系统能力与交互布局的控制优势，另一方面利用高德地图 Web API 成熟的地图与导航能力。

层次	对应文件/能力	主要职责
UI 层	index.ets	登录页、主界面、底部抽屉、Tab 分栏、按钮交互
业务控制层	Index 组件方法	权限申请、实时轮询、桥接 JS 调用、状态同步
地图与分析层	amap.html + AMap API	地图初始化、POI 搜索、路线规划、GNSS 七项增强
配置资源层	module.json5 / string.json	权限声明、入口配置、资源字符串与权限理由

3.2 UI 界面设计

主界面采用全屏地图 + 底部抽屉式面板的设计。地图作为视觉中心置于最底层，底部面板提供收起、半展开和全展开三种高度状态，并以“首页—导航—GNSS—工具”四个页签进行功能分栏。

首页聚合搜索、定位当前位置、拍照定位与卫星图切换；导航页聚合起点、终点和路线模式；GNSS 页以卡片方式展示七项增强功能；工具页则集中放置实时模式、围栏、轨迹与拍照等辅助按钮。



图 3-1 登录界面截图（包含演示账号与密码提示）



图 3-2 主界面首页截图（地图 + 抽屉面板 + 搜索栏）

3.3 功能模块设计

3.3.1 登录模块设计

doLogin 方法校验演示账号密码，合法后切换至主界面。

该模块采用轻量设计，不接入数据库与服务器。

3.3.2 地图基础模块设计

Web 组件通过 rawfile 加载 amap.html，网页端 initAMap 创建地图实例并加载控件。

状态提示统一由 setStatus/showAppToast 完成。

3.3.3 定位与实时动态模块设计

定位模块由 requestBasicPermissions 与 getCurrentPosition 协同实现。

实时动态模块通过 startRealtimeMode、stopRealtimeMode 与 pollLatestLocationOnce 完成定时轮询和刷新。

3.3.4 地点搜索模块设计

鸿蒙端对关键词转义后调用 `searchPlace`，网页端调用 `PlaceSearch` 检索 POI 并标记。

3.3.5 路线规划模块设计

`routeMode` 记录模式，网页端 `planRoute` 选择不同路径服务。

3.3.6 电子围栏模块设计

`addFence/removeFence` 与 `drawFenceFromWGS84/removeFence` 成对实现围栏绘制与删除。

3.3.7 轨迹绘制模块设计

`TrackPoint` 数组既支持手动添加，也支持实时模式自动追加。

3.3.8 拍照定位模块设计

`handlePhotoLocation` 同时依赖当前位置与 `CameraKit` 拍照结果。

3.3.9 GNSS 七项增强模块设计

卡片入口经 `openGnssTool` 与网页端 `openGnssTool(tool)` 联动，页面按工具编号动态渲染。

该设计便于后续继续扩展更多 GNSS 工具。

3.4 权限设计

权限名	来源文件	作用	说明
<code>ohos.permission.APPROXIMATELY_LOCATION</code>	<code>module.json5</code>	获取大致位置	用于定位与实时位置更新
<code>ohos.permission.LOCATION</code>	<code>module.json5</code>	获取精确位置	用于当前位置、导航与 GNSS 分析
<code>ohos.permission.CAMERA</code>	<code>module.json5</code>	拍照与拍照定位	配合 <code>CameraKit</code> 使用
<code>ohos.permission.INTERNET</code>	<code>module.json5</code>	访问高德地图与	地图脚本、POI 搜索与路

string.json 中进一步为位置与相机权限提供了中文说明文字，使权限申请更符合用户理解习惯。

3.5 数据流程设计

3.5.1 定位与实时刷新数据流程

轮询获取位置后，鸿蒙端同时更新地图、轨迹和 GNSS 实时页面。

3.5.2 路线规划数据流程

采集起终点与模式后，网页端完成路线绘制与面板展示。

3.5.3 拍照定位与围栏/轨迹数据流程

拍照定位依赖相机与位置，围栏与轨迹依赖连续位置样本。

四、代码实现

4.1 核心配置文件实现

module.json5 定义了 entry 模块的主能力 EntryAbility、设备类型、页面配置与请求权限。相较于基础版，本版本显式保留了 INTERNET 权限，以保证高德地图脚本和路径规划接口可正常访问。

string.json 对模块描述、入口说明、应用标签与权限说明进行统一资源化管理，使权限申请更规范、后续维护更方便。

4.2 登录与主界面代码实现

index.ets 开头引入 AbilityKit、LocationKit、CameraKit 与 WebView 模块，并定义 TrackPoint 类存储轨迹点。

build 方法根据 isLoggedIn 决定渲染 LoginPage 还是 MainPage；MainPage 以 Stack 容器叠加 Web 地图和底部面板。

4.3 地图、定位与搜索代码实现

getCurrentPosition 在更新经纬度后，会调用 pushGnssObservationToWeb 将样本传递给网页端。

doSearch 对搜索关键字转义后调用 searchPlace；网页端使用 PlaceSearch 返回 POI 并定位。

4.4 路线规划、围栏与轨迹实现

路线规划由 `routeMode` 和 `doNavigate` 发起，网页端 `planRoute` 完成三种模式切换。

围栏与轨迹分别通过圆形和折线进行绘制，实时模式下轨迹可自动追加。

4.5 拍照定位实现

`handlePhotoLocation` 在完成拍照后，将照片 URI 与当前位置绑定并传给网页端相机标记。

4.6 GNSS 七项增强实现

坐标转换由多套坐标变换函数组成；精度评估和误差分析依赖最近样本统计。

质量评估解析 NMEA 文本；组合导航、欺骗检测与 CORS/PPP 分别使用融合、异常分析和模拟改正函数。

4.7 实时动态模式实现

鸿蒙端使用 `isRealtimeMode` 和 `realtimeTimer` 管理动态轮询，网页端使用 `refreshGnssRealtimePanels` 按当前面板实时重绘。

4.8 典型问题调试与修复

本次开发经历了权限声明格式、签名过期、高德脚本加载失败、Web 面板遮挡、函数未绑定等问题，并已形成对应修复策略。

问题类型	表现	原因分析	解决办法
权限配置错误	<code>module.json5</code> 报格式错误	<code>usedScene / reason</code> 理解不一致	按对象结构补齐 <code>usedScene</code> 与 <code>reason</code>
签名证书过期	Build 停在 SignHap	旧证书失效	重新生成或托管签名
高德脚本加载失败	地图空白	key、安全密钥、INTERNET 权限或网络问题	校验 key、 <code>securityJsCode</code> 与联网状态
GNSS 面板点击无反应	关闭按钮和表单不可点	原生控件遮挡或 JS 未绑定	移除遮挡层并统一暴露 <code>window</code> 方法
实时模式前期样本不足	组合导航页面只显示提示	统计量需要多样本	开启实时模式累积样本

五、系统测试

5.1 测试目的与环境

测试目的在于验证基础导航功能与 GNSS 实时增强功能是否达到预期，包括功能正确性、交互可用性和持续运行稳定性。

测试以鸿蒙真机为主，结合实际室外/校园环境进行定位与实时动态观察，并通过截图、面板指标与状态提示验证各模块运行结果。

5.2 功能测试

测试项	测试步骤	预期结果	结论
登录	输入预设账号密码并点击登录	进入主界面，无报错	通过
定位	点击“定位当前位置”	地图跳转到当前位置，显示蓝色圆形标记	通过
搜索	输入关键字并搜索	地图定位到目标地点	通过
导航	分别执行驾车/骑行/步行模式	路线与面板详情正常显示	通过
围栏	添加后再删除围栏	圆形围栏正常绘制与清除	通过
轨迹	手动添加或开启实时模式	轨迹线正常延伸，可清空	通过
拍照定位	完成拍摄	地图生成相机标记位置、轨迹与	通过
实时动态	开启实时模式 20 秒	GNSS 面板自动刷新	通过
坐标转换	输入坐标并转换	表单返回结果	通过
精度评估	开启实时模式后进入页面	样本数、RMS、CEP95 动态刷新	通过
欺骗检测	进入页面	风险等级和触发因素正常生成	通过
CORS/PPP	应用改正结果到地图	显示改正前后对比与 C 点标记	通过

5.3 性能测试

性能测试主要关注地图加载时间、定位响应时间、实时模式下轮询是否稳定以及 GNSS 面板刷新是否造成明显卡顿。结果表明，在课堂演示级别的数据量下，实时轮询与动态刷新能够保持良好流畅性。

性能瓶颈主要集中在网络请求与 Web 端渲染开销，而非鸿蒙原生层本身，因此后续优化应重点关注脚本加载缓存、轨迹点数量上限和图层重绘频率控制。

5.4 兼容性与安全性测试

系统围绕鸿蒙手机竖屏场景进行适配，底部抽屉式面板在收起、半展开和全展开状态下均能正确显示。

系统仅申请必要权限；照片 URI、定位结果与登录信息均仅用于本地演示，不上传服务器，也未设计持久化存储。

5.5 测试结果总结

综合功能测试与真机交互验证，本版本 APP 已能够稳定完成基础导航与 GNSS 实时增强两大类任务。

当前 CORS / PPP 仍属于课程设计级模拟实现，组合导航也采用轻量时间窗平滑，但它们已经在界面层与交互层完成了教学演示所需的主要表达。

六、优化方向

6.1 功能优化

后续可增加登录状态持久化、常用地点收藏、历史搜索记录与路线收藏等功能。

拍照定位与轨迹记录可引入本地数据库或文件持久化机制。

6.2 性能优化

可对实时动态模式下的刷新频率进行自适应调整，以平衡耗电、网络与实时性。

网页端对于长轨迹与长样本序列的处理可加入截断与降采样策略。

6.3 UI 与交互优化

后续可引入真实拖拽手势与动画过渡，并为 GNSS 页面设计更接近专业仪表盘的布局。

还可增加深色模式、自适应字体和更完整的错误引导。

6.4 GNSS 能力深化方向

下一步可接入原始观测数据、GNSS 硬件测量接口或外部接收机数据流。

在增强定位方面，可进一步接入 NTRIP/RTCM 服务实现真实 CORS 差分，或接入 PPP 后端服务实现更可信的改正试验。

七、总结与展望

7.1 开发总结

本次开发在基础版导航 APP 框架上，完成了实时动态 GNSS 增强版本的设计与实现。系统既保留了登录、地图、定位、搜索、路线规划、围栏、轨迹和拍照定位等原有能力，又新增了七项面向 GNSS 课程教学的动态分析功能，使 APP 从“基础导航应用”升级为“导航 + GNSS 实验平台”。

从代码层看，index.ets 与 amap.html 的分层协同、module.json5 与 string.json 的资源化配置，以及对签名、权限、高德脚本加载和交互冲突问题的持续修复，共同构成了本次开发最有价值的实践经验。

7.2 未来展望

未来，本项目可继续朝移动应用方向增强产品化能力，如离线地图、语音播报、多设备协同和数据持久化；也可沿 GNSS 实验方向增强专业性，如原始测量值接入、INS/GNSS 紧组合、真实差分改正与多源数据融合。

若能进一步将当前 APP 与课程实验、校园定位场景乃至外业采集任务结合起来，该系统将不仅是一项课程作业成果，也可能发展为具有持续使用价值的教学辅助平台。

附录 A module.json5 完整代码

本附录收录 module.json5 的完整代码，用于展示鸿蒙应用入口、权限与资源引用配置。

A.1 module.json5 清单

以下代码清单来自文件“module.json5(3).txt”。

```
{
  "module": {
    "name": "entry",
    "type": "entry",
    "description": "$string:module_desc",
    "mainElement": "EntryAbility",
    "deviceTypes": [
      "phone"
    ],
    "deliveryWithInstall": true,
    "installationFree": false,
    "pages": "$profile:main_pages",
    "abilities": [
      {
        "name": "EntryAbility",
        "srcEntry": "./ets/entryability/EntryAbility.ets",
        "description": "$string:EntryAbility_desc",
        "icon": "$media:layered_image",
        "label": "$string:EntryAbility_label",
        "startWindowIcon": "$media:startIcon",
        "startWindowBackground": "$color:start_window_background",
        "exported": true,
        "skills": [
          {
            "entities": [
              "entity.system.home"
            ],
            "actions": [
              "ohos.want.action.home"
            ]
          }
        ]
      }
    ],
    "extensionAbilities": [
      {
        "name": "EntryBackupAbility",
        "srcEntry": "./ets/entrybackupability/EntryBackupAbility.ets",
        "type": "backup",
        "exported": false,
        "metadata": [
          {
            "name": "ohos.extension.backup",
            "resource": "$profile:backup_config"
          }
        ]
      }
    ],
    "requestPermissions": [
      {
        "name": "ohos.permission.APPROXIMATELY_LOCATION",
        "reason": "$string:approx_location_permission_reason",
        "usedScene": {
          "abilities": [
            "EntryAbility"
          ],
          "when": "inuse"
        }
      },
      {
        "name": "ohos.permission.LOCATION",
        "reason": "$string:location_permission_reason",
```

```

        "usedScene": {
            "abilities": [
                "EntryAbility"
            ],
            "when": "inuse"
        }
    },
    {
        "name": "ohos.permission.CAMERA",
        "reason": "$string:camera_permission_reason",
        "usedScene": {
            "abilities": [
                "EntryAbility"
            ],
            "when": "inuse"
        }
    },
    {
        "name": "ohos.permission.INTERNET"
    }
]
}
}

```

附录 B string.json 完整代码

本附录收录 string.json 的完整代码，用于展示模块描述、应用标签与权限中文说明配置。

B.1 string.json 清单

以下代码清单来自文件“string.json(3).txt”。

```
{
  "string": [
    {
      "name": "module_desc",
      "value": "基于 HarmonyOS 的卫星导航与 GNSS 增强 APP 模块"
    },
    {
      "name": "EntryAbility_desc",
      "value": "卫星导航 APP 主入口能力"
    },
    {
      "name": "EntryAbility_label",
      "value": "鸿蒙导航定位 APP"
    },
    {
      "name": "approx_location_permission_reason",
      "value": "需要获取大致位置信息，用于定位功能"
    },
    {
      "name": "location_permission_reason",
      "value": "需要获取精确位置信息，用于显示当前位置、导航与 GNSS 分析"
    },
    {
      "name": "camera_permission_reason",
      "value": "需要使用相机拍照，实现拍照定位功能"
    }
  ]
}
```


附录 C index.ets 完整代码

本附录收录 index.ets 的完整代码。该文件是鸿蒙端界面与业务控制的核心，实现了登录、抽屉面板、路线规划入口、工具页、GNSS 卡片页和实时动态模式。

C.1 index.ets 清单

以下代码清单来自文件“index.ets(3).txt”。

```
import { abilityAccessCtrl, common, Permissions } from '@kit.AbilityKit'
import { geoLocationManager } from '@kit.LocationKit'
import { camera, cameraPicker } from '@kit.CameraKit'
import web_webview from '@ohos.web.webview'

class TrackPoint {
    lat: number
    lng: number

    constructor(lat: number, lng: number) {
        this.lat = lat
        this.lng = lng
    }
}

@Entry
@Component
struct Index {
    @State account: string = ''
    @State password: string = ''
    @State loginError: string = ''
    @State isLoggedIn: boolean = false

    @State latitude: string = '未获取'
    @State longitude: string = '未获取'
    @State searchKeyword: string = ''
    @State isSatellite: boolean = false

    @State photoUri: string = '未拍照'
    @State photoTime: string = '未记录'

    @State startKeyword: string = ''
    @State endKeyword: string = ''
    @State routeMode: string = 'driving'

    @State activeTab: string = 'home'
    @State panelLevel: number = 1 // 0=收起 1=半展开 2=全展开
    @State isRealtimeMode: boolean = false

    private webController: web_webview.WebviewController = new web_webview.WebviewController()
    private trackPoints: Array<TrackPoint> = []
    private realtimeTimer: number = -1

    private readonly DEMO_ACCOUNT: string = '15541547005'
    private readonly DEMO_PASSWORD: string = 'qwe20050308'

    aboutToDisappear() {
        this.stopRealtimeMode(false)
    }

    private getPanelHeight(): number {
        if (this.panelLevel === 0) {
            return 120
        }
        if (this.panelLevel === 1) {
            return 390
        }
        return 680
    }

    private async requestBasicPermissions() {
```

```

let context = getContext(this) as common.UIAbilityContext
let atManager = abilityAccessCtrl.createAtManager()

let permissions: Permissions[] = [
  'ohos.permission.APPROXIMATELY_LOCATION',
  'ohos.permission.LOCATION',
  'ohos.permission.CAMERA'
]

return await atManager.requestPermissionsFromUser(context, permissions)
}

private async getCurrentPosition(silent: boolean = false): Promise<boolean> {
  try {
    let permissionResult = await this.requestBasicPermissions()

    if (!permissionResult || !permissionResult.authResults ||
    permissionResult.authResults.length < 2) {
      if (!silent) {
        this.showToast('权限申请结果异常')
      }
      return false
    }

    let approxGranted = permissionResult.authResults[0] === 0
    let preciseGranted = permissionResult.authResults[1] === 0

    if (!approxGranted || !preciseGranted) {
      if (!silent) {
        this.showToast('定位权限被拒绝, 请在系统设置中开启')
      }
      return false
    }

    const request: geoLocationManager.SingleLocationRequest = {
      locatingPriority: geoLocationManager.LocatingPriority.PRIORITY_LOCATING_SPEED,
      locatingTimeoutMs: 10000
    }

    const location = await geoLocationManager.getCurrentLocation(request)
    this.latitude = String(location.latitude)
    this.longitude = String(location.longitude)

    this.pushGnssObservationToWeb(Number(this.longitude), Number(this.latitude))

    if (!silent) {
      this.showToast('定位成功')
    }
    return true
  } catch (error) {
    if (!silent) {
      this.showToast(`定位失败: ${JSON.stringify(error)}`)
    }
    return false
  }
}

private pushGnssObservationToWeb(lng: number, lat: number) {
  this.runMapJs(`window.pushGnssObservation && window.pushGnssObservation(${lng}, ${lat},
${Date.now()});`)
  this.runMapJs(`window.refreshGnssRealtimePanels && window.refreshGnssRealtimePanels();`)
}

private async pollLatestLocationOnce(pushToMap: boolean = true, silent: boolean = true) {
  let ok = await this.getCurrentPosition(silent)
  if (!ok) {
    return
  }

  const lat = Number(this.latitude)
  const lng = Number(this.longitude)

  if (pushToMap) {
    this.runMapJs(`showCurrentLocationFromWGS84(${lng}, ${lat});`)
  }
}

```

```

    if (this.trackPoints.length === 0) {
      this.trackPoints.push(new TrackPoint(lat, lng))
    } else {
      const last = this.trackPoints[this.trackPoints.length - 1]
      if (Math.abs(last.lat - lat) > 1e-9 || Math.abs(last.lng - lng) > 1e-9) {
        this.trackPoints.push(new TrackPoint(lat, lng))
      }
    }

    if (this.isRealtimeMode && this.trackPoints.length > 1) {
      let pointsJs = '[' + this.trackPoints.map(item => `[${item.lng},
${item.lat}]`).join(',') + ']'
      this.runMapJs(`drawTrackFromWGS84(${pointsJs});`)
    }

    this.runMapJs(`window.refreshGnssRealtimePanels && window.refreshGnssRealtimePanels();`)
  }

  private startRealtimeMode() {
    if (this.isRealtimeMode) {
      return
    }

    this.isRealtimeMode = true
    this.showToast('实时动态模式已开启')
    this.pollLatestLocationOnce(true, true)

    this.realtimeTimer = setInterval(() => {
      this.pollLatestLocationOnce(true, true)
    }, 2000) as number
  }

  private stopRealtimeMode(showMsg: boolean = true) {
    if (!this.isRealtimeMode && this.realtimeTimer === -1) {
      return
    }

    this.isRealtimeMode = false

    if (this.realtimeTimer !== -1) {
      clearInterval(this.realtimeTimer)
      this.realtimeTimer = -1
    }

    if (showMsg) {
      this.showToast('实时动态模式已关闭')
    }
  }

  private toggleRealtimeMode() {
    if (this.isRealtimeMode) {
      this.stopRealtimeMode(true)
    } else {
      this.startRealtimeMode()
    }
  }

  private doLogin() {
    if (this.account === this.DEMO_ACCOUNT && this.password === this.DEMO_PASSWORD) {
      this.loginError = ''
      this.isLoggedIn = true
      return
    }
    this.loginError = '账号或密码错误'
  }

  private showToast(msg: string) {
    this.runMapJs(`showAppToast('${msg.replace(/'/g, '\\\'')}');`)
  }

  private runMapJs(jsCode: string) {
    try {
      this.webController.runJavaScript(jsCode)
    } catch (error) {
      console.error('地图脚本执行失败: ${JSON.stringify(error)}')
    }
  }

```

```

    }

    private async locateCurrentOnMap() {
        let ok = await this.getCurrentPosition(false)
        if (!ok) {
            return
        }

        const lat = Number(this.latitude)
        const lng = Number(this.longitude)
        this.runMapJs(`showCurrentLocationFromWGS84(${lng}, ${lat});`)
    }

    private async doSearch() {
        if (!this.searchKeyword.trim()) {
            this.showToast('请输入地点关键字')
            return
        }

        const safeKeyword = this.searchKeyword.replace(/'/g, "\\'")
        this.runMapJs(`searchPlace('${safeKeyword}');`)
    }

    private toggleSatellite() {
        this.isSatellite = !this.isSatellite
        this.runMapJs(`toggleSatellite(${this.isSatellite ? 'true' : 'false'});`)
        this.showToast(this.isSatellite ? '已切换到卫星图' : '已切换到标准图')
    }

    private async addFence() {
        let ok = await this.getCurrentPosition(true)
        if (!ok) {
            this.showToast('围栏添加失败: 无法获取当前位置')
            return
        }

        const lat = Number(this.latitude)
        const lng = Number(this.longitude)
        const radius = 200

        this.runMapJs(`drawFenceFromWGS84(${lng}, ${lat}, ${radius});`)
        this.showToast('围栏添加成功')
    }

    private removeFence() {
        this.runMapJs(`removeFence();`)
        this.showToast('围栏已删除')
    }

    private async addTrack() {
        let ok = await this.getCurrentPosition(true)
        if (!ok) {
            this.showToast('轨迹添加失败: 无法获取当前位置')
            return
        }

        const lat = Number(this.latitude)
        const lng = Number(this.longitude)
        this.trackPoints.push(new TrackPoint(lat, lng))

        let pointsJs = '[' + this.trackPoints.map(item => `[${item.lng}, ${item.lat}]`).join(',') + ']'
        this.runMapJs(`drawTrackFromWGS84(${pointsJs});`)
        this.showToast(`已添加轨迹点: ${this.trackPoints.length} 个`)
    }

    private clearTrack() {
        this.trackPoints = []
        this.runMapJs(`clearTrack();`)
        this.showToast('轨迹已清除')
    }

    private async handlePhotoLocation() {
        try {
            let ok = await this.getCurrentPosition(true)

```

```

    if (!ok) {
      this.showToast('拍照定位失败: 无法获取当前位置')
      return
    }

    const pickerProfile: cameraPicker.PickerProfile = {
      cameraPosition: camera.CameraPosition.CAMERA_POSITION_BACK
    }

    const pickerResult: cameraPicker.PickerResult = await cameraPicker.pick(
      this.getUIContext().getHostContext(),
      [cameraPicker.PickerMediaType.PHOTO],
      pickerProfile
    )

    if (!pickerResult || !pickerResult.resultUri) {
      this.showToast('用户取消拍照或未返回照片')
      return
    }

    this.photoUri = pickerResult.resultUri
    this.photoTime = new Date().toLocaleString()

    const lat = Number(this.latitude)
    const lng = Number(this.longitude)
    const safeUri = this.photoUri.replace(/'/g, "\\'")

    this.runMapJs(`addPhotoMarkerFromWGS84(${lng}, ${lat}, '${safeUri}');`)
    this.showToast('拍照定位成功')
  } catch (error) {
    this.showToast(`拍照定位失败: ${JSON.stringify(error)}`)
  }
}

private setRouteMode(mode: string) {
  this.routeMode = mode
}

private doNavigate() {
  if (!this.startKeyword.trim() || !this.endKeyword.trim()) {
    this.showToast('请输入起点和终点')
    return
  }

  const safeStart = this.startKeyword.replace(/'/g, "\\'")
  const safeEnd = this.endKeyword.replace(/'/g, "\\'")
  const safeMode = this.routeMode.replace(/'/g, "\\'")
  this.runMapJs(`planRoute('${safeStart}', '${safeEnd}', '${safeMode}');`)
}

private clearRoute() {
  this.runMapJs(`clearRoute();`)
  this.showToast('路线已清除')
}

private openGnssTool(tool: string) {
  this.activeTab = 'gnss'
  this.panelLevel = 0
  this.runMapJs(`openGnssTool('${tool}');`)
}

private async openGnssToolWithCurrent(tool: string) {
  await this.getCurrentPosition(true)
  this.activeTab = 'gnss'
  this.panelLevel = 0
  this.runMapJs(`openGnssTool('${tool}');`)
}

private cyclePanelLevel() {
  if (this.panelLevel === 0) {
    this.panelLevel = 1
    return
  }
  if (this.panelLevel === 1) {
    this.panelLevel = 2
  }
}

```

```

        return
    }
    this.panelLevel = 0
}

private expandPanel() {
    if (this.panelLevel < 2) {
        this.panelLevel += 1
    }
}

private collapsePanel() {
    if (this.panelLevel > 0) {
        this.panelLevel -= 1
    }
}

private switchTab(tab: string) {
    this.activeTab = tab
    if (tab !== 'gnss') {
        this.runMapJs(`closeGnssTool();`)
    }
    if (this.panelLevel === 0) {
        this.panelLevel = 1
    }
}

@Builder
TabButton(label: string, value: string) {
    Button(label)
        .layoutWeight(1)
        .height(38)
        .borderRadius(12)
        .backgroundColor(this.activeTab === value ? '#1E80FF' : '#F2F4F7')
        .fontColor(this.activeTab === value ? Color.White : '#344054')
        .onClick(() => {
            this.switchTab(value)
        })
}

@Builder
PrimaryAction(label: string, action: () => void) {
    Button(label)
        .layoutWeight(1)
        .height(46)
        .borderRadius(14)
        .backgroundColor('#1E80FF')
        .fontColor(Color.White)
        .onClick(() => {
            action()
        })
}

@Builder
LightAction(label: string, action: () => void) {
    Button(label)
        .layoutWeight(1)
        .height(42)
        .borderRadius(14)
        .backgroundColor('#EEF4FF')
        .fontColor('#1E80FF')
        .onClick(() => {
            action()
        })
}

@Builder
DangerAction(label: string, action: () => void) {
    Button(label)
        .layoutWeight(1)
        .height(42)
        .borderRadius(14)
        .backgroundColor('#FFF1F0')
        .fontColor('#D4380D')
        .onClick(() => {
            action()
        })
}

```

```

    }

    @Builder
    ToolCard(title: string, desc: string, action: () => void) {
        Column({ space: 8 }) {
            Text(title)
                .fontSize(18)
                .fontWeight(FontWeight.Medium)
                .fontColor('#101828')
                .width('100%')
                .textAlign(TextAlign.Start)

            Text(desc)
                .fontSize(12)
                .fontColor('#667085')
                .width('100%')
                .textAlign(TextAlign.Start)
                .maxLines(2)
        }
        .width('48%')
        .height(88)
        .padding(12)
        .backgroundColor(Color.White)
        .borderRadius(16)
        .shadow({ radius: 10, color: '#11000000', offsetX: 0, offsetY: 4 })
        .onClick(() => {
            action()
        })
    }

    @Builder
    HomeTab() {
        Column({ space: 12 }) {
            Row({ space: 10 }) {
                TextInput({ placeholder: '搜索地点 / POI / 地址', text: this.searchKeyword })
                    .layoutWeight(1)
                    .height(46)
                    .backgroundColor('#F4F6FA')
                    .borderRadius(12)
                    .onChange((value: string) => {
                        this.searchKeyword = value
                    })

                Button('搜索')
                    .height(46)
                    .borderRadius(12)
                    .onClick(() => {
                        this.doSearch()
                    })
            }

            Row({ space: 8 }) {
                this.PrimaryAction('定位当前位置', () => {
                    this.locateCurrentOnMap()
                })

                this.PrimaryAction(this.isRealtimeMode ? '关闭实时' : '开启实时', () => {
                    this.toggleRealtimeMode()
                })
            }

            Row({ space: 8 }) {
                this.LightAction('拍照定位', () => {
                    this.handlePhotoLocation()
                })

                this.LightAction(this.isSatellite ? '标准图' : '卫星图', () => {
                    this.toggleSatellite()
                })
            }

            if (this.panelLevel === 2) {
                Column({ space: 10 }) {
                    Text('实时状态')
                        .fontSize(16)

```

```

        .fontWeight(FontWeight.Medium)
        .width('100%')
        .textAlign(TextAlign.Start)

Row({ space: 8 }) {
    Column({ space: 4 }) {
        Text('经度')
            .fontSize(12)
            .fontColor('#667085')
        Text(this.longitude)
            .fontSize(14)
            .fontWeight(FontWeight.Medium)
    }
    .layoutWeight(1)
    .padding(12)
    .backgroundColor('#F8FAFC')
    .borderRadius(14)

    Column({ space: 4 }) {
        Text('纬度')
            .fontSize(12)
            .fontColor('#667085')
        Text(this.latitude)
            .fontSize(14)
            .fontWeight(FontWeight.Medium)
    }
    .layoutWeight(1)
    .padding(12)
    .backgroundColor('#F8FAFC')
    .borderRadius(14)
}

Row({ space: 8 }) {
    Column({ space: 4 }) {
        Text('实时模式')
            .fontSize(12)
            .fontColor('#667085')
        Text(this.isRealtimeMode ? '已开启' : '未开启')
            .fontSize(13)
    }
    .layoutWeight(1)
    .padding(12)
    .backgroundColor('#F8FAFC')
    .borderRadius(14)

    Column({ space: 4 }) {
        Text('照片状态')
            .fontSize(12)
            .fontColor('#667085')
        Text(this.photoUri == '未拍照' ? '未拍照' : '已记录')
            .fontSize(13)
    }
    .layoutWeight(1)
    .padding(12)
    .backgroundColor('#F8FAFC')
    .borderRadius(14)
}
}
}
}

@Builder
NavTab() {
    Column({ space: 12 }) {
        TextInput({ placeholder: '起点', text: this.startKeyword })
            .width('100%')
            .height(44)
            .backgroundColor('#F4F6FA')
            .borderRadius(12)
            .onChange((value: string) => {
                this.startKeyword = value
            })

        TextInput({ placeholder: '终点', text: this.endKeyword })
    }
}

```



```

        .width('100%')
        .height(44)
        .backgroundColor('#F4F6FA')
        .borderRadius(12)
        .onChange((value: string) => {
            this.endKeyword = value
        })

Row({ space: 8 }) {
    Button('驾车')
        .layoutWeight(1)
        .height(40)
        .borderRadius(12)
        .backgroundColor(this.routeMode === 'driving' ? '#1E80FF' : '#EAF2FF')
        .fontColor(this.routeMode === 'driving' ? Color.White : '#1E80FF')
        .onClick(() => {
            this.setRouteMode('driving')
        })

    Button('骑行')
        .layoutWeight(1)
        .height(40)
        .borderRadius(12)
        .backgroundColor(this.routeMode === 'riding' ? '#1E80FF' : '#EAF2FF')
        .fontColor(this.routeMode === 'riding' ? Color.White : '#1E80FF')
        .onClick(() => {
            this.setRouteMode('riding')
        })

    Button('步行')
        .layoutWeight(1)
        .height(40)
        .borderRadius(12)
        .backgroundColor(this.routeMode === 'walking' ? '#1E80FF' : '#EAF2FF')
        .fontColor(this.routeMode === 'walking' ? Color.White : '#1E80FF')
        .onClick(() => {
            this.setRouteMode('walking')
        })
}

Row({ space: 8 }) {
    this.PrimaryAction('开始导航', () => {
        this.doNavigate()
    })

    this.DangerAction('清除路线', () => {
        this.clearRoute()
    })
}
}
}

@Builder
GnssTab() {
    Column({ space: 10 }) {
        Row({ space: 10 }) {
            this.ToolCard('坐标转换', 'WGS84 / GCJ02 / BD09 互转', () => {
                this.openGnssTool('transform')
            })
            this.ToolCard('精度评估', '散布、均值误差、RMS、CEP95', () => {
                this.openGnssToolWithCurrent('precision')
            })
        }

        Row({ space: 10 }) {
            this.ToolCard('质量评估', '卫星数、SNR、DOP 解析', () => {
                this.openGnssTool('quality')
            })
            this.ToolCard('误差分析', '多路径、几何、噪声综合分析', () => {
                this.openGnssToolWithCurrent('errors')
            })
        }

        Row({ space: 10 }) {
            this.ToolCard('组合导航', 'GNSS + 惯性平滑实验', () => {

```

```

        this.openGnssTool('fusion')
    })
    this.ToolCard('欺骗检测', '位置突跳、速度异常、弱信号检测', () => {
        this.openGnssToolWithCurrent('spoof')
    })
}

Row({ space: 10 }) {
    this.ToolCard('CORS / PPP', '差分改正与长时间窗平滑实验', () => {
        this.openGnssTool('correction')
    })
    this.ToolCard('GNSS 总览', '实时刷新所有 GNSS 工具统计', () => {
        this.openGnssTool('overview')
    })
}
}

@Builder
ToolsTab() {
    Column({ space: 12 }) {
        Row({ space: 8 }) {
            this.PrimaryAction('定位', () => {
                this.locateCurrentOnMap()
            })

            this.PrimaryAction(this.isRealtimeMode ? '关闭实时' : '开启实时', () => {
                this.toggleRealtimeMode()
            })
        }

        Row({ space: 8 }) {
            this.LightAction('添加围栏', () => {
                this.addFence()
            })

            this.DangerAction('删除围栏', () => {
                this.removeFence()
            })

            this.LightAction(this.isSatellite ? '标准图' : '卫星图', () => {
                this.toggleSatellite()
            })
        }

        Row({ space: 8 }) {
            this.LightAction('添加轨迹', () => {
                this.addTrack()
            })

            this.DangerAction('清空轨迹', () => {
                this.clearTrack()
            })
        }

        Row({ space: 8 }) {
            this.LightAction('拍照定位', () => {
                this.handlePhotoLocation()
            })
        }
    }
}

@Builder
PanelContent() {
    if (this.activeTab === 'home') {
        this.HomeTab()
    } else if (this.activeTab === 'nav') {
        this.NavTab()
    } else if (this.activeTab === 'gnss') {
        this.GnssTab()
    } else {
        this.ToolsTab()
    }
}

```

```

    }

    @Builder
    LoginPage() {
      Column({ space: 20 }) {
        Blank().height('10%')

        Text('鸿蒙导航定位 APP')
          .fontSize(30)
          .fontWeight(FontWeight.Bold)

        Column({ space: 16 }) {
          TextInput({ placeholder: '请输入账号', text: this.account })
            .width('100%')
            .height(52)
            .type(InputType.PhoneNumber)
            .backgroundColor('#F6F7FB')
            .borderRadius(14)
            .onChange((value: string) => {
              this.account = value
            })

          TextInput({ placeholder: '请输入密码', text: this.password })
            .width('100%')
            .height(52)
            .type(InputType.Password)
            .backgroundColor('#F6F7FB')
            .borderRadius(14)
            .onChange((value: string) => {
              this.password = value
            })

          Button('登录')
            .width('100%')
            .height(52)
            .borderRadius(14)
            .onClick(() => {
              this.doLogin()
            })

          if (this.loginError) {
            Text(this.loginError)
              .fontColor('#E84026')
              .fontSize(14)
          }
        }
        .width('86%')
        .padding(20)
        .backgroundColor(Color.White)
        .borderRadius(20)
        .shadow({ radius: 20, color: '#1A000000', offsetX: 0, offsetY: 8 })

        Text('演示账号: 15541547005\n 演示密码: qwe20050308')
          .fontSize(13)
          .fontColor('#666666')
          .textAlign(TextAlign.Center)
      }
      .width('100%')
      .height('100%')
      .justifyContent(FlexAlign.Center)
      .backgroundColor('#EEF3FF')
    }

    @Builder
    MainPage() {
      Stack({ alignContent: Alignment.Bottom }) {
        Web({
          src: $rawfile('amap.html'),
          controller: this.webController
        })
          .width('100%')
          .height('100%')

        Column({ space: 12 }) {
          Row() {
            Blank()

```

```

        .width(48)
        .height(5)
        .borderRadius(3)
        .backgroundColor('#D0D5DD')
        .onClick(() => {
            this.cyclePanelLevel()
        })
    }
    .justifyContent(FlexAlign.Center)
    .width('100%')
    .margin({ top: 6 })

Row({ space: 8 }) {
    Button('-')
        .width(44)
        .height(28)
        .borderRadius(10)
        .backgroundColor('#F2F4F7')
        .fontColor('#344054')
        .onClick(() => {
            this.collapsePanel()
        })

    Button(this.panelLevel === 0 ? '已收起' : this.panelLevel === 1 ? '半展开' : '全展开'
    ')
        .layoutWeight(1)
        .height(28)
        .borderRadius(10)
        .backgroundColor('#F8FAFC')
        .fontColor('#475467')
        .onClick(() => {
            this.cyclePanelLevel()
        })

    Button('+')
        .width(44)
        .height(28)
        .borderRadius(10)
        .backgroundColor('#F2F4F7')
        .fontColor('#344054')
        .onClick(() => {
            this.expandPanel()
        })
}

Row({ space: 8 }) {
    this.TabButton('首页', 'home')
    this.TabButton('导航', 'nav')
    this.TabButton('GNSS', 'gnss')
    this.TabButton('工具', 'tools')
}

if (this.panelLevel === 0) {
    Row({ space: 8 }) {
        TextInput({ placeholder: '搜索地点 / 去哪里', text: this.searchKeyword })
            .layoutWeight(1)
            .height(44)
            .backgroundColor('#F4F6FA')
            .borderRadius(12)
            .onChange((value: string) => {
                this.searchKeyword = value
            })

        Button('搜索')
            .height(44)
            .borderRadius(12)
            .onClick(() => {
                this.doSearch()
            })
    }
} else {
    Scroll() {
        Column({ space: 14 }) {
            this.PanelContent()
        }
    }
}

```

```

        .width('100%')
    }
    .width('100%')
    .layoutWeight(1)
    .scrollBar(BarState.Off)
  }
}
.width('100%')
.height(this.getPanelHeight())
.padding({ left: 16, right: 16, top: 8, bottom: 20 })
.backgroundColor(Color.White)
.borderRadius({ topLeft: 28, topRight: 28 })
.shadow({ radius: 18, color: '#22000000', offsetX: 0, offsetY: -4 })
}
.width('100%')
.height('100%')
}

build() {
  if (!this.isLoggedIn) {
    this.LoginPage()
  } else {
    this.MainPage()
  }
}
}
}

```

附录 D amap.html 完整代码

本附录收录 amap.html 的完整代码。该文件承担高德地图渲染、坐标转换、路线规划、围栏/轨迹、GNSS 七项增强与实时统计刷新等逻辑。

D.1 amap.html 清单

以下代码清单来自文件“amap.html(2).txt”。

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>AMap Realtime GNSS</title>
  <meta name="viewport" content="width=device-width, initial-scale=1.0,maximum-
scale=1.0,user-scalable=no" />
  <style>
    html, body, #container {
      width: 100%;
      height: 100%;
      margin: 0;
      padding: 0;
      background: #f5f5f5;
    }

    .info-card {
      position: absolute;
      top: 12px;
      left: 12px;
      z-index: 999;
      background: rgba(255,255,255,0.94);
      padding: 10px 12px;
      border-radius: 12px;
      font-size: 13px;
      color: #333;
      box-shadow: 0 2px 10px rgba(0,0,0,0.12);
      max-width: 300px;
      line-height: 1.5;
      white-space: pre-line;
      display: none;
    }

    #routePanel {
      position: absolute;
      right: 12px;
      top: 90px;
      z-index: 999;
      width: 260px;
      max-height: 320px;
      overflow: auto;
      background: rgba(255,255,255,0.96);
      border-radius: 12px;
      padding: 10px;
      box-shadow: 0 2px 10px rgba(0,0,0,0.12);
      font-size: 12px;
      display: none;
    }

    #gnssPanel {
      position: absolute;
      left: 12px;
      right: 12px;
      top: 76px;
      bottom: 132px;
      z-index: 999;
      overflow: auto;
      background: rgba(255,255,255,0.98);
      border-radius: 18px;
      padding: 16px;
      box-shadow: 0 4px 18px rgba(0,0,0,0.16);
      display: none;
    }
```

```

        box-sizing: border-box;
    }

    .gnss-title {
        font-size: 16px;
        font-weight: bold;
        margin-bottom: 10px;
    }

    .gnss-box {
        padding: 12px;
        background: #F8FAFC;
        border-radius: 12px;
        line-height: 1.8;
        color: #111827;
    }

    .gnss-row {
        display: flex;
        gap: 10px;
        margin-bottom: 10px;
        flex-wrap: wrap;
    }

    .gnss-input, .gnss-select, .gnss-textarea {
        flex: 1;
        padding: 12px;
        border-radius: 12px;
        border: 1px solid #D0D5DD;
        box-sizing: border-box;
        font-size: 14px;
        background: white;
        color: #111827;
        min-width: 120px;
    }

    .gnss-textarea {
        width: 100%;
        min-height: 120px;
        resize: vertical;
    }

    .btn-primary {
        flex: 1;
        padding: 12px;
        border: none;
        border-radius: 12px;
        background: #1E80FF;
        color: white;
        font-size: 14px;
        min-width: 120px;
    }

    .btn-light {
        flex: 1;
        padding: 12px;
        border: none;
        border-radius: 12px;
        background: #EEF4FF;
        color: #1E80FF;
        font-size: 14px;
        min-width: 120px;
    }

    .btn-danger {
        padding: 10px 16px;
        border: none;
        border-radius: 12px;
        background: #FCEFEA;
        color: #C2410C;
        font-size: 14px;
    }
</style>
</head>
<body>
<div id="container"></div>
<div class="info-card" id="statusBox"></div>

```

```

<div id="routePanel"></div>

<div id="gnssPanel">
  <div style="display:flex;justify-content:space-between;align-items:center;margin-
bottom:12px;">
    <div style="font-size:18px;font-weight:bold;">GNSS 综合增强 (实时动态) </div>
    <button onclick="closeGnssTool()" class="btn-danger">关闭</button>
  </div>
  <div id="gnssToolContent" style="font-size:14px;color:#333;"></div>
</div>

<script>
function setStatus(text) {
  const box = document.getElementById('statusBox');
  if (box) {
    box.innerText = text;
    box.style.display = 'block';
    clearTimeout(window.__toastTimer);
    window.__toastTimer = setTimeout(function() {
      box.style.display = 'none';
    }, 1800);
  }
}

function showAppToast(text) {
  setStatus(text);
}

window.onerror = function(message, source, lineno) {
  setStatus('JS 错误: ' + message + '\n 来源: ' + source + '\n 行号: ' + lineno);
};

function loadAMapFailed() {
  setStatus('高德脚本加载失败, 请检查 WebKey、securityJsCode、INTERNET 权限和网络');
}

function outOfChina(lng, lat) {
  return !(lng > 73.66 && lng < 135.05 && lat > 3.86 && lat < 53.55);
}

function transformLat(x, y) {
  let ret = -100.0 + 2.0 * x + 3.0 * y + 0.2 * y * y + 0.1 * x * y + 0.2 *
Math.sqrt(Math.abs(x));
  ret += (20.0 * Math.sin(6.0 * x * Math.PI) + 20.0 * Math.sin(2.0 * x * Math.PI)) * 2.0
/ 3.0;
  ret += (20.0 * Math.sin(y * Math.PI) + 40.0 * Math.sin(y / 3.0 * Math.PI)) * 2.0 /
3.0;
  ret += (160.0 * Math.sin(y / 12.0 * Math.PI) + 320.0 * Math.sin(y * Math.PI / 30.0)) *
2.0 / 3.0;
  return ret;
}

function transformLng(x, y) {
  let ret = 300.0 + x + 2.0 * y + 0.1 * x * x + 0.1 * x * y + 0.1 *
Math.sqrt(Math.abs(x));
  ret += (20.0 * Math.sin(6.0 * x * Math.PI) + 20.0 * Math.sin(2.0 * x * Math.PI)) * 2.0
/ 3.0;
  ret += (20.0 * Math.sin(x * Math.PI) + 40.0 * Math.sin(x / 3.0 * Math.PI)) * 2.0 /
3.0;
  ret += (150.0 * Math.sin(x / 12.0 * Math.PI) + 300.0 * Math.sin(x / 30.0 * Math.PI)) *
2.0 / 3.0;
  return ret;
}

function wgs84ToGcj02(lng, lat) {
  if (outOfChina(lng, lat)) {
    return { lng: lng, lat: lat };
  }
  let a = 6378245.0;
  let ee = 0.00669342162296594323;
  let dLat = transformLat(lng - 105.0, lat - 35.0);
  let dLng = transformLng(lng - 105.0, lat - 35.0);
  let radLat = lat / 180.0 * Math.PI;
  let magic = Math.sin(radLat);
  magic = 1 - ee * magic * magic;
  let sqrtMagic = Math.sqrt(magic);

```



```

    dLat = (dLat * 180.0) / ((a * (1 - ee)) / (magic * sqrtMagic) * Math.PI);
    dLng = (dLng * 180.0) / (a / sqrtMagic * Math.cos(radLat) * Math.PI);
    let mgLat = lat + dLat;
    let mgLng = lng + dLng;
    return { lng: mgLng, lat: mgLat };
}

function gcj02ToWgs84(lng, lat) {
    let g = wgs84ToGcj02(lng, lat);
    return { lng: lng * 2 - g.lng, lat: lat * 2 - g.lat };
}

function gcj02ToBd09(lng, lat) {
    let z = Math.sqrt(lng * lng + lat * lat) + 0.00002 * Math.sin(lat * Math.PI * 3000.0 / 180.0);
    let theta = Math.atan2(lat, lng) + 0.000003 * Math.cos(lng * Math.PI * 3000.0 / 180.0);
    return { lng: z * Math.cos(theta) + 0.0065, lat: z * Math.sin(theta) + 0.006 };
}

function bd09ToGcj02(lng, lat) {
    let x = lng - 0.0065;
    let y = lat - 0.006;
    let z = Math.sqrt(x * x + y * y) - 0.00002 * Math.sin(y * Math.PI * 3000.0 / 180.0);
    let theta = Math.atan2(y, x) - 0.000003 * Math.cos(x * Math.PI * 3000.0 / 180.0);
    return { lng: z * Math.cos(theta), lat: z * Math.sin(theta) };
}

function toRad(v) {
    return v * Math.PI / 180.0;
}

function calcDistanceMeters(lng1, lat1, lng2, lat2) {
    const R = 6378137;
    const dLat = toRad(lat2 - lat1);
    const dLng = toRad(lng2 - lng1);
    const a =
        Math.sin(dLat / 2) * Math.sin(dLat / 2) +
        Math.cos(toRad(lat1)) * Math.cos(toRad(lat2)) *
        Math.sin(dLng / 2) * Math.sin(dLng / 2);
    const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
    return R * c;
}

function calcHeadingDeg(lng1, lat1, lng2, lat2) {
    const y = Math.sin(toRad(lng2 - lng1)) * Math.cos(toRad(lat2));
    const x =
        Math.cos(toRad(lat1)) * Math.sin(toRad(lat2)) -
        Math.sin(toRad(lat1)) * Math.cos(toRad(lat2)) * Math.cos(toRad(lng2 - lng1));
    let brng = Math.atan2(y, x) * 180 / Math.PI;
    brng = (brng + 360) % 360;
    return brng;
}

function mean(arr) {
    return arr.length ? arr.reduce((a, b) => a + b, 0) / arr.length : null;
}

function percentile(arr, p) {
    if (!arr.length) return null;
    const s = arr.slice().sort((a, b) => a - b);
    const idx = Math.min(s.length - 1, Math.max(0, Math.round((s.length - 1) * p)));
    return s[idx];
}

let map = null;
let currentMarker = null;
let fenceCircle = null;
let trackPolyline = null;
let photoMarkers = [];
let satelliteLayer = null;
let roadNetLayer = null;
let driving = null;
let riding = null;
let walking = null;

const gnssState = {

```

```

    observations: [],
    rawObservations: [],
    activeTool: '',
    quality: { visible: null, avgCn0: null, maxCn0: null, pdop: null, hdop: null, vdop:
null },
    correctionMode: 'OFF',
    correctionOffset: { dlng: 0, dlat: 0 },
    pppWindow: [],
    fusionEnabled: false,
    fusionState: { vx: 0, vy: 0 },
    anomaly: [],
    nmeaText: ''
  });

function getRecentObservations(limit) {
  if (!gnssState.observations.length) {
    return [];
  }
  return gnssState.observations.slice(-limit);
}

function buildCurrentMarker(position, titleText) {
  if (currentMarker) {
    map.remove(currentMarker);
  }

  currentMarker = new AMap.Marker({
    position: position,
    anchor: 'bottom-center',
    offset: new AMap.Pixel(0, 0),
    content:
      '<div style="position:relative;width:24px;height:24px;">' +
      '<div style="position:absolute;left:2px;top:2px;width:20px;height:20px;border-
radius:50%;background:#1E80FF;border:3px solid rgba(255,255,255,0.95);box-shadow:0 0 0 10px
rgba(30,128,255,0.18);"></div>' +
      '</div>',
    title: titleText || '当前位置'
  });

  map.add(currentMarker);
}

function computeScatterStats(points) {
  if (!points || points.length < 2) {
    return null;
  }

  const meanLng = points.reduce((s, p) => s + p.lng, 0) / points.length;
  const meanLat = points.reduce((s, p) => s + p.lat, 0) / points.length;
  const ds = points.map(p => calcDistanceMeters(p.lng, p.lat, meanLng, meanLat));
  const avg = ds.reduce((a, b) => a + b, 0) / ds.length;
  const rms = Math.sqrt(ds.reduce((a, b) => a + b * b, 0) / ds.length);
  const cep95 = percentile(ds, 0.95);

  return { meanLng, meanLat, avg, rms, cep95 };
}

function computeMotionStats(points) {
  if (!points || points.length < 2) {
    return null;
  }

  let maxSpeed = 0;
  let speedSamples = [];
  let headingChanges = [];

  for (let i = 1; i < points.length; i++) {
    const p1 = points[i - 1];
    const p2 = points[i];
    const dist = calcDistanceMeters(p1.lng, p1.lat, p2.lng, p2.lat);
    const dt = Math.max(0.5, (p2.ts - p1.ts) / 1000.0);
    const speed = dist / dt;
    speedSamples.push(speed);
    maxSpeed = Math.max(maxSpeed, speed);

    if (i >= 2) {

```

```

        const p0 = points[i - 2];
        const h1 = calcHeadingDeg(p0.lng, p0.lat, p1.lng, p1.lat);
        const h2 = calcHeadingDeg(p1.lng, p1.lat, p2.lng, p2.lat);
        let diff = Math.abs(h2 - h1);
        if (diff > 180) {
            diff = 360 - diff;
        }
        headingChanges.push(diff);
    }
}

return {
    avgSpeed: mean(speedSamples) || 0,
    maxSpeed,
    avgHeadingChange: mean(headingChanges) || 0
};
}

function applyCorrectionWgs(lng, lat) {
    if (gnssState.correctionMode === 'CORS') {
        return [lng + gnssState.correctionOffset.dlng, lat +
gnssState.correctionOffset.dlat];
    }

    if (gnssState.correctionMode === 'PPP') {
        gnssState.pppWindow.push([lng, lat]);
        if (gnssState.pppWindow.length > 12) {
            gnssState.pppWindow.shift();
        }
        const xs = gnssState.pppWindow.map(v => v[0]);
        const ys = gnssState.pppWindow.map(v => v[1]);
        return [mean(xs), mean(ys)];
    }

    return [lng, lat];
}

function fuseWithMotion(lng, lat, ts) {
    if (!gnssState.fusionEnabled || !gnssState.observations.length) {
        return [lng, lat];
    }

    const last = gnssState.observations[gnssState.observations.length - 1];
    const dt = Math.max(0.5, (ts - last.ts) / 1000.0);
    const fusedLng = 0.78 * lng + 0.22 * (last.lng + gnssState.fusionState.vx * dt /
111320.0);
    const fusedLat = 0.78 * lat + 0.22 * (last.lat + gnssState.fusionState.vy * dt /
110540.0);
    return [fusedLng, fusedLat];
}

function updateVelocity() {
    if (gnssState.observations.length < 2) {
        return;
    }

    const a = gnssState.observations[gnssState.observations.length - 2];
    const b = gnssState.observations[gnssState.observations.length - 1];
    const dt = Math.max(0.5, (b.ts - a.ts) / 1000.0);
    gnssState.fusionState.vx = (b.lng - a.lng) * 111320.0 / dt;
    gnssState.fusionState.vy = (b.lat - a.lat) * 110540.0 / dt;
}

function detectAnomaly(rawObs) {
    const notes = [];
    const n = gnssState.rawObservations.length;

    if (n >= 2) {
        const prev = gnssState.rawObservations[n - 2];
        const dist = calcDistanceMeters(prev.lng, prev.lat, rawObs.lng, rawObs.lat);
        const dt = Math.max(0.5, (rawObs.ts - prev.ts) / 1000.0);
        const speed = dist / dt;
        if (speed > 65) {
            notes.push('瞬时速度异常偏高');
        }
        if (dist > 80 && dt < 2) {

```

```

        notes.push('短时位置突跳');
    }
}

if (gnssState.quality.hdop !== null && gnssState.quality.hdop > 6) {
    notes.push('HDOP 偏大');
}
if (gnssState.quality.avgCn0 !== null && gnssState.quality.avgCn0 < 25) {
    notes.push('平均 C/N0 偏低');
}
if (gnssState.quality.visible !== null && gnssState.quality.visible < 5) {
    notes.push('可见卫星数不足');
}

gnssState.anomaly = notes;
}

function renderMetricCard(label, value, unit, color) {
    return `
        <div style="flex:1;min-width:110px;padding:12px;border-
radius:14px;background:#F8FAFC;border:1px solid #E5E7EB;box-sizing:border-box;">
            <div style="font-size:12px;color:#667085;margin-bottom:6px;">${label}</div>
            <div style="font-size:20px;font-weight:bold;color:${color || '#111827'};">
                ${value}<span style="font-size:12px;font-weight:normal;margin-left:4px;">${unit
|| ''}</span>
            </div>
        </div>
    `;
}

function computeSpoofAnalysis(points) {
    if (!points || points.length < 2) {
        return {
            level: '低',
            score: 0,
            summary: '样本不足，建议先开启实时模式采集连续样本。',
            jumpCount: 0,
            maxJump: 0,
            maxSpeed: 0,
            avgHeadingChange: 0,
            rms: 0,
            reasons: ['样本不足']
        };
    }

    const motion = computeMotionStats(points);
    const scatter = computeScatterStats(points);
    let jumpCount = 0;
    let maxJump = 0;
    let score = 0;
    let reasons = [];

    for (let i = 1; i < points.length; i++) {
        const d = calcDistanceMeters(points[i - 1].lng, points[i - 1].lat, points[i].lng,
points[i].lat);
        maxJump = Math.max(maxJump, d);
        if (d > 30) {
            jumpCount += 1;
        }
    }

    if (jumpCount >= 2) {
        score += 35;
        reasons.push('连续位置突跳较多');
    } else if (jumpCount === 1) {
        score += 18;
        reasons.push('检测到单次位置突跳');
    }

    if (motion && motion.maxSpeed > 20) {
        score += 25;
        reasons.push('短时间速度异常偏高');
    } else if (motion && motion.maxSpeed > 10) {
        score += 12;
    }
}

```

```

    reasons.push('速度变化偏大');
  }

  if (motion && motion.avgHeadingChange > 90) {
    score += 20;
    reasons.push('航向突变明显');
  }

  if (scatter && scatter.rms > 15) {
    score += 20;
    reasons.push('定位散布较大');
  } else if (scatter && scatter.rms > 8) {
    score += 10;
    reasons.push('定位稳定性一般');
  }

  let level = '低';
  if (score >= 60) {
    level = '高';
  } else if (score >= 30) {
    level = '中';
  }

  if (reasons.length === 0) {
    reasons.push('未发现明显异常模式');
  }

  return {
    level,
    score,
    summary: level === '高'
      ? '存在较强异常/欺骗风险, 建议暂停信任当前解算结果。'
      : level === '中'
        ? '存在一定异常风险, 建议继续观察并结合质量评估结果判断。'
        : '当前未发现明显异常, 定位样本整体较稳定。',
    jumpCount,
    maxJump,
    maxSpeed: motion ? motion.maxSpeed : 0,
    avgHeadingChange: motion ? motion.avgHeadingChange : 0,
    rms: scatter ? scatter.rms : 0,
    reasons
  };
}

function computeFusionResult(points) {
  if (!points || points.length < 2) {
    return null;
  }

  const windowSize = Math.min(points.length, 8);
  const recent = points.slice(-windowSize);
  const smoothLng = recent.reduce((s, p) => s + p.lng, 0) / recent.length;
  const smoothLat = recent.reduce((s, p) => s + p.lat, 0) / recent.length;
  const latest = recent[recent.length - 1];
  const motion = computeMotionStats(recent);
  const scatter = computeScatterStats(recent);

  let heading = 0;
  let speed = 0;
  if (recent.length >= 2) {
    const p1 = recent[recent.length - 2];
    const p2 = recent[recent.length - 1];
    heading = calcHeadingDeg(p1.lng, p1.lat, p2.lng, p2.lat);
    const dt = Math.max(0.5, (p2.ts - p1.ts) / 1000.0);
    speed = calcDistanceMeters(p1.lng, p1.lat, p2.lng, p2.lat) / dt;
  }

  return {
    latestLng: latest.lng,
    latestLat: latest.lat,
    fusedLng: smoothLng,
    fusedLat: smoothLat,
    heading,
    speed,
  };
}

```

```

        windowSize,
        scatterRms: scatter ? scatter.rms : 0,
        avgSpeed: motion ? motion.avgSpeed : 0
    };
}

function computeCorrectionStats(points) {
    if (!points || points.length < 3) {
        return null;
    }

    const rawStats = computeScatterStats(points);
    if (!rawStats) {
        return null;
    }

    const recent = points.slice(-Math.min(points.length, 10));
    const fusedLng = recent.reduce((s, p) => s + p.lng, 0) / recent.length;
    const fusedLat = recent.reduce((s, p) => s + p.lat, 0) / recent.length;

    const correctedPoints = points.map((p, idx) => {
        const alpha = idx < points.length - recent.length ? 0.7 : 0.45;
        return {
            lng: p.lng * alpha + fusedLng * (1 - alpha),
            lat: p.lat * alpha + fusedLat * (1 - alpha)
        };
    });

    const corrStats = computeScatterStats(correctedPoints);
    if (!corrStats) {
        return null;
    }

    const improveRms = rawStats.rms > 0
        ? ((rawStats.rms - corrStats.rms) / rawStats.rms) * 100
        : 0;

    return {
        rawMeanLng: rawStats.meanLng,
        rawMeanLat: rawStats.meanLat,
        rawRms: rawStats.rms,
        rawCep95: rawStats.cep95,
        corrMeanLng: corrStats.meanLng,
        corrMeanLat: corrStats.meanLat,
        corrRms: corrStats.rms,
        corrCep95: corrStats.cep95,
        improveRms
    };
}

function pushGnssObservation(lng, lat, ts) {
    const rawObs = { lng, lat, ts };
    gnssState.rawObservations.push(rawObs);
    if (gnssState.rawObservations.length > 200) {
        gnssState.rawObservations.shift();
    }

    let corrected = applyCorrectionWgs(lng, lat);
    corrected = fuseWithMotion(corrected[0], corrected[1], ts);

    const obs = { lng: corrected[0], lat: corrected[1], ts };
    gnssState.observations.push(obs);
    if (gnssState.observations.length > 200) {
        gnssState.observations.shift();
    }

    if (gnssState.observations.length >= 2) {
        const a = gnssState.observations[gnssState.observations.length - 2];
        const b = gnssState.observations[gnssState.observations.length - 1];
        const dt = Math.max(0.5, (b.ts - a.ts) / 1000.0);
        gnssState.fusionState.vx = (b.lng - a.lng) * 111320.0 / dt;
        gnssState.fusionState.vy = (b.lat - a.lat) * 110540.0 / dt;
    }

    detectAnomaly(rawObs);
}

```

```

function refreshGnssRealtimePanels() {
  if (gnssState.activeTool) {
    const panel = document.getElementById('gnssPanel');
    if (panel && panel.style.display !== 'none') {
      openGnssTool(gnssState.activeTool);
    }
  }
}

function showFusedPointOnMap() {
  const result = computeFusionResult(getRecentObservations(20));
  if (!result) {
    setStatus('暂无足够定位样本');
    return;
  }

  const pos = wgs84ToGcj02(result.fusedLng, result.fusedLat);
  const marker = new AMap.Marker({
    position: [pos.lng, pos.lat],
    anchor: 'bottom-center',
    content:
      '<div style="position:relative;width:28px;height:38px;">' +
      '<div style="width:28px;height:28px;border-
radius:50%;background:#7C3AED;color:white;display:flex;align-items:center;justify-
content:center;font-size:13px;box-shadow:0 3px 8px rgba(0,0,0,0.25);">F</div>' +
      '</div>',
    title: '组合导航融合点'
  });
  map.add(marker);
  map.setZoomAndCenter(18, [pos.lng, pos.lat]);
  setStatus('已显示融合结果点');
}

function showCorrectedPointOnMap() {
  const result = computeCorrectionStats(getRecentObservations(20));
  if (!result) {
    setStatus('样本不足，无法应用改正');
    return;
  }

  const pos = wgs84ToGcj02(result.corrMeanLng, result.corrMeanLat);
  const marker = new AMap.Marker({
    position: [pos.lng, pos.lat],
    anchor: 'bottom-center',
    content:
      '<div style="position:relative;width:28px;height:38px;">' +
      '<div style="width:28px;height:28px;border-
radius:50%;background:#16A34A;color:white;display:flex;align-items:center;justify-
content:center;font-size:13px;box-shadow:0 3px 8px rgba(0,0,0,0.25);">C</div>' +
      '</div>',
    title: '改正后结果点'
  });
  map.add(marker);
  map.setZoomAndCenter(18, [pos.lng, pos.lat]);
  setStatus('已显示改正后结果点');
}

function enableCorsMode() {
  const points = getRecentObservations(20);
  if (points.length < 3) {
    setStatus('样本不足，无法启用 CORS 模拟');
    return;
  }
  const stats = computeScatterStats(points);
  const latest = points[points.length - 1];
  gnssState.correctionMode = 'CORS';
  gnssState.correctionOffset.dlng = stats.meanLng - latest.lng;
  gnssState.correctionOffset.dlat = stats.meanLat - latest.lat;
  setStatus('已启用 CORS 模拟改正');
  refreshGnssRealtimePanels();
}

function enablePppMode() {
  gnssState.correctionMode = 'PPP';
  gnssState.pppWindow = [];
}

```

```

    setStatus('已启用 PPP 平滑模式');
    refreshGnssRealtimePanels();
}

function disableCorrectionMode() {
    gnssState.correctionMode = 'OFF';
    gnssState.pppWindow = [];
    gnssState.correctionOffset = { dlng: 0, dlat: 0 };
    setStatus('已关闭改正模式');
    refreshGnssRealtimePanels();
}

function toggleFusionMode() {
    gnssState.fusionEnabled = !gnssState.fusionEnabled;
    setStatus(gnssState.fusionEnabled ? '组合导航已开启' : '组合导航已关闭');
    openGnssTool('fusion');
}

function parseNmeaQuality() {
    const text = document.getElementById('nmeaInput').value || '';
    gnssState.nmeaText = text;
    const lines = text.split(/\r?\n/);
    let satCount = 0;
    let snrs = [];
    let pdop = null;
    let hdop = null;
    let vdop = null;

    lines.forEach(line => {
        if (line.includes('GSV')) {
            const parts = line.split(',');
            for (let i = 7; i < parts.length; i += 4) {
                const snr = parseFloat(parts[i]);
                if (!isNaN(snr)) {
                    snrs.push(snr);
                    satCount += 1;
                }
            }
        }
        if (line.includes('GSA')) {
            const parts = line.split(',');
            pdop = parseFloat(parts[15]) || pdop;
            hdop = parseFloat(parts[16]) || hdop;
            vdop = parseFloat(parts[17]) || vdop;
        }
    });

    gnssState.quality.visible = satCount;
    gnssState.quality.avgCn0 = snrs.length ? mean(snrs) : null;
    gnssState.quality.maxCn0 = snrs.length ? Math.max(...snrs) : null;
    gnssState.quality.pdop = pdop;
    gnssState.quality.hdop = hdop;
    gnssState.quality.vdop = vdop;

    setStatus('NMEA 质量解析完成');
    refreshGnssRealtimePanels();
}

function closeGnssTool() {
    const panel = document.getElementById('gnssPanel');
    if (panel) {
        panel.style.display = 'none';
    }
    gnssState.activeTool = '';
}

function openGnssTool(tool) {
    const panel = document.getElementById('gnssPanel');
    const content = document.getElementById('gnssToolContent');

    if (!panel || !content) {
        setStatus('GNSS 面板未初始化');
        return;
    }

    gnssState.activeTool = tool;

```



```

panel.style.display = 'block';

if (tool === 'transform') {
  const latest = gnssState.rawObservations.length ?
  gnssState.rawObservations[gnssState.rawObservations.length - 1] : null;
  content.innerHTML = `
    <div class="gnss-title">1. 坐标转换 (实时) </div>
    <div class="gnss-row">
      <input id="convLng" class="gnss-input" placeholder="经度" value="${latest ?
latest.lng : ''}">
      <input id="convLat" class="gnss-input" placeholder="纬度" value="${latest ?
latest.lat : ''}">
    </div>
    <div class="gnss-row">
      <select id="convFrom" class="gnss-select">
        <option>WGS84</option>
        <option>GCJ02</option>
        <option>BD09</option>
      </select>
      <select id="convTo" class="gnss-select">
        <option>GCJ02</option>
        <option>WGS84</option>
        <option>BD09</option>
      </select>
    </div>
    <div class="gnss-row">
      <button class="btn-primary" onclick="runCoordTransform()">转换</button>
      <button class="btn-light" onclick="fillLatestCoord()">取最新定位</button>
    </div>
    <div id="coordResult" class="gnss-box">最近实时样本: ${latest ?
`${latest.lng.toFixed(7)}, ${latest.lat.toFixed(7)}` : '暂无'}</div>
  `;
  return;
}

if (tool === 'precision') {
  const stats = computeScatterStats(getRecentObservations(20));
  if (!stats) {
    content.innerHTML = `
      <div class="gnss-title">2. GNSS 定位精度评估 (实时) </div>
      <div class="gnss-box">当前暂无足够定位样本, 建议先开启实时模式或多点击几次“定位当前位置”。
    </div>
  `;
  return;
}

content.innerHTML = `
  <div class="gnss-title">2. GNSS 定位精度评估 (实时) </div>
  <div class="gnss-row">
    ${renderMetricCard('样本数', getRecentObservations(20).length, '个', '#111827')}
    ${renderMetricCard('平均散布', stats.avg.toFixed(2), 'm', '#2563EB')}
  </div>
  <div class="gnss-row">
    ${renderMetricCard('RMS', stats.rms.toFixed(2), 'm', '#7C3AED')}
    ${renderMetricCard('CEP95', (stats.cep95 || 0).toFixed(2), 'm', '#DC2626')}
  </div>
  <div class="gnss-box">
    均值坐标: ${stats.meanLng.toFixed(7)}, ${stats.meanLat.toFixed(7)}<br>
    本页面会随实时定位样本自动刷新, 无需反复手动定位。
  </div>
  `;
  return;
}

if (tool === 'quality') {
  content.innerHTML = `
    <div class="gnss-title">3. GNSS 定位质量评估</div>
    <textarea id="nmeaInput" class="gnss-textarea" placeholder="粘贴 GSA / GSV NMEA 语
句">${gnssState.nmeaText || ''}</textarea>
    <div class="gnss-row">
      <button class="btn-primary" onclick="parseNmeaQuality()">解析质量</button>
      <button class="btn-light" onclick="refreshGnssRealtimePanels()">刷新面板</button>
    </div>
  `;

```

```

        <div class="gnss-row">
            ${renderMetricCard('可见卫星数', gnssState.quality.visible != null ?
gnssState.quality.visible : '--', '个', '#2563EB')}
            ${renderMetricCard('平均 C/N0', gnssState.quality.avgCn0 != null ?
gnssState.quality.avgCn0.toFixed(1) : '--', 'dB-Hz', '#16A34A')}
        </div>
        <div class="gnss-row">
            ${renderMetricCard('PDOP', gnssState.quality.pdop != null ?
gnssState.quality.pdop.toFixed(2) : '--', '', '#7C3AED')}
            ${renderMetricCard('HDOP', gnssState.quality.hdop != null ?
gnssState.quality.hdop.toFixed(2) : '--', '', '#D97706')}
        </div>
        <div class="gnss-box">
            最大 C/N0: ${gnssState.quality.maxCn0 != null ?
gnssState.quality.maxCn0.toFixed(1) + ' dB-Hz' : '暂无'}<br>
            VDOP: ${gnssState.quality.vdop != null ? gnssState.quality.vdop.toFixed(2) : '暂
无'}
        </div>
    `;
    return;
}

if (tool === 'errors') {
    const points = getRecentObservations(20);
    const stats = computeScatterStats(points);
    const motion = computeMotionStats(points);
    const text = stats && stats.rms > 15
        ? '近期散布偏大, 优先怀疑多路径或遮挡。'
        : '近期位置散布较平稳。';

    content.innerHTML = `
        <div class="gnss-title">4. GNSS 导航误差源分析 (实时) </div>
        <div class="gnss-row">
            ${renderMetricCard('散布 RMS', stats ? stats.rms.toFixed(2) : 'N/A', 'm',
'#2563EB')}
            ${renderMetricCard('平均速度', motion ? motion.avgSpeed.toFixed(2) : 'N/A',
'm/s', '#16A34A')}
        </div>
        <div class="gnss-row">
            ${renderMetricCard('航向变化', motion ? motion.avgHeadingChange.toFixed(1) :
'N/A', '°', '#D97706')}
            ${renderMetricCard('样本数', points.length, '个', '#111827')}
        </div>
        <div class="gnss-box">
            <b>误差源解释:</b><br>
            1. 多路径误差: 楼宇、树木密集区域更明显。<br>
            2. 卫星几何误差: DOP 偏大时精度下降。<br>
            3. 遮挡与信号衰减: 可见卫星数不足时更明显。<br>
            4. 接收机噪声: 会引入短时抖动。<br><br>
            <b>当前判断:</b><br>${text}
        </div>
    `;
    return;
}

if (tool === 'fusion') {
    const result = computeFusionResult(getRecentObservations(20));
    if (!result) {
        content.innerHTML = `
            <div class="gnss-title">5. GNSS + 惯性组合导航 (实时) </div>
            <div class="gnss-box">样本不足, 建议先开启实时模式后再观察融合结果。</div>
        `;
        return;
    }
}

content.innerHTML = `
    <div class="gnss-title">5. GNSS + 惯性组合导航 (实时) </div>
    <div class="gnss-row">
        ${renderMetricCard('平滑窗口', result.windowSize, '点', '#111827')}
        ${renderMetricCard('瞬时速度', result.speed.toFixed(2), 'm/s', '#16A34A')}
    </div>
    <div class="gnss-row">

```

```

    ${renderMetricCard('航向角', result.heading.toFixed(1), '°', '#2563EB')}
    ${renderMetricCard('平滑 RMS', result.scatterRms.toFixed(2), 'm', '#7C3AED')}
  </div>
  <div class="gnss-box" style="margin-bottom:10px;">
    <b>原始最新点: </b><br>
    ${result.latestLng.toFixed(7)}, ${result.latestLat.toFixed(7)}<br><br>
    <b>融合平滑点: </b><br>
    ${result.fusedLng.toFixed(7)}, ${result.fusedLat.toFixed(7)}<br><br>
    开启后会随时定位样本自动更新融合结果。
  </div>
  <div class="gnss-row">
    <button class="${gnssState.fusionEnabled ? 'btn-light' : 'btn-primary'}"
style="${gnssState.fusionEnabled ? 'color:#D4380D;background:#FFF1F0;' : ''}"
onclick="toggleFusionMode()">
      ${gnssState.fusionEnabled ? '关闭组合导航' : '开启组合导航'}
    </button>
    <button class="btn-light" onclick="showFusedPointOnMap()">显示融合点</button>
  </div>
`;
  return;
}

if (tool === 'spoof') {
  const analysis = computeSpoofAnalysis(getRecentObservations(20));
  const levelColor = analysis.level === '高' ? '#DC2626' : analysis.level === '中' ?
'#D97706' : '#16A34A';

  content.innerHTML = `
    <div class="gnss-title">6. GNSS 欺骗与异常检测 (实时) </div>
    <div class="gnss-row">
      ${renderMetricCard('风险等级', analysis.level, '', levelColor)}
      ${renderMetricCard('风险分数', analysis.score.toFixed(0), '分', levelColor)}
    </div>
    <div class="gnss-row">
      ${renderMetricCard('最大突跳', analysis.maxJump.toFixed(2), 'm', '#2563EB')}
      ${renderMetricCard('最高速度', analysis.maxSpeed.toFixed(2), 'm/s', '#7C3AED')}
    </div>
    <div class="gnss-row">
      ${renderMetricCard('突跳次数', analysis.jumpCount, '次', '#111827')}
      ${renderMetricCard('航向突变', analysis.avgHeadingChange.toFixed(1), '°',
'#D97706')}
    </div>
    <div class="gnss-box">
      <b>诊断结论: </b><br>${analysis.summary}<br><br>
      <b>触发因素: </b><br>${analysis.reasons.map((r, i) => `${i + 1}.
${r}`).join('<br>')}
    </div>
  `;
  return;
}

if (tool === 'correction') {
  const stats = computeCorrectionStats(getRecentObservations(20));
  if (!stats) {
    content.innerHTML = `
      <div class="gnss-title">7. CORS / PPP 提升定位精度 (实时) </div>
      <div class="gnss-box">样本不足, 建议先开启实时模式采集更多定位样本后再进行改正评估。</div>
    `;
    return;
  }

  content.innerHTML = `
    <div class="gnss-title">7. CORS / PPP 提升定位精度 (实时) </div>
    <div class="gnss-row">
      ${renderMetricCard('改正前 RMS', stats.rawRms.toFixed(2), 'm', '#DC2626')}
      ${renderMetricCard('改正后 RMS', stats.corrRms.toFixed(2), 'm', '#16A34A')}
    </div>
    <div class="gnss-row">
      ${renderMetricCard('RMS 改善', stats.improveRms.toFixed(1), '%', '#2563EB')}
      ${renderMetricCard('改正后 CEP95', (stats.corrCep95 || 0).toFixed(2), 'm',
'#7C3AED')}
    </div>
  `;

```

```

<div class="gnss-box" style="margin-bottom:10px;">
  <b>改正前均值: </b><br>
  ${stats.rawMeanLng.toFixed(7)}, ${stats.rawMeanLat.toFixed(7)}<br><br>
  <b>改正后均值: </b><br>
  ${stats.corrMeanLng.toFixed(7)}, ${stats.corrMeanLat.toFixed(7)}<br><br>
  当前为实时实验版: CORS 使用控制量偏移模拟改正; PPP 使用时间窗平滑模拟精密单点改进。
</div>
<div class="gnss-row">
  <button class="btn-primary" onclick="enableCorsMode()">启用 CORS 模拟</button>
  <button class="btn-light" onclick="enablePppMode()">启用 PPP 平滑</button>
</div>
<div class="gnss-row">
  <button class="btn-light" onclick="disableCorrectionMode()">关闭改正</button>
  <button class="btn-light" onclick="showCorrectedPointOnMap()">显示改正点</button>
</div>
`;
return;
}

content.innerHTML = `
<div class="gnss-title">GNSS 总览 (实时) </div>
<div class="gnss-box">
  当前实时样本数: ${getRecentObservations(50).length}<br>
  当前改正模式: ${gnssState.correctionMode}<br>
  组合导航: ${gnssState.fusionEnabled ? '已开启' : '未开启'}<br>
  异常提示: ${gnssState.anomaly.length ? gnssState.anomaly.join(',') : '暂无'}<br><br>
  开启实时模式后, 本页面和各 GNSS 工具页会自动刷新, 无需每次手动定位。
</div>
`;
}

function fillLatestCoord() {
  if (!gnssState.rawObservations.length) {
    setStatus('暂无定位样本');
    return;
  }
  const p = gnssState.rawObservations[gnssState.rawObservations.length - 1];
  document.getElementById('convLng').value = p.lng;
  document.getElementById('convLat').value = p.lat;
}

function runCoordTransform() {
  const lng = parseFloat(document.getElementById('convLng').value);
  const lat = parseFloat(document.getElementById('convLat').value);
  const from = document.getElementById('convFrom').value;
  const to = document.getElementById('convTo').value;
  const box = document.getElementById('coordResult');

  if (isNaN(lng) || isNaN(lat)) {
    box.innerHTML = '请输入有效经纬度';
    return;
  }

  let res = { lng: lng, lat: lat };

  if (from === to) {
    box.innerHTML = `转换结果: ${lng.toFixed(7)}, ${lat.toFixed(7)}`;
    return;
  }

  if (from === 'WGS84' && to === 'GCJ02') {
    res = wgs84ToGcj02(lng, lat);
  } else if (from === 'GCJ02' && to === 'WGS84') {
    res = gcj02ToWgs84(lng, lat);
  } else if (from === 'GCJ02' && to === 'BD09') {
    res = gcj02ToBd09(lng, lat);
  } else if (from === 'BD09' && to === 'GCJ02') {
    res = bd09ToGcj02(lng, lat);
  } else if (from === 'WGS84' && to === 'BD09') {
    const gcj = wgs84ToGcj02(lng, lat);
    res = gcj02ToBd09(gcj.lng, gcj.lat);
  } else if (from === 'BD09' && to === 'WGS84') {

```

```

        const gcj = bd09ToGcj02(lng, lat);
        res = gcj02ToWgs84(gcj.lng, gcj.lat);
    }

    box.innerHTML = `转换结果: ${res.lng.toFixed(7)}, ${res.lat.toFixed(7)}`;
}

function showCurrentLocationFromWGS84(lng, lat) {
    pushGnssObservation(lng, lat, Date.now());
    const last = gnssState.observations[gnssState.observations.length - 1] || { lng,
lat };
    const gcj = wgs84ToGcj02(last.lng, last.lat);
    map.setZoomAndCenter(17, [gcj.lng, gcj.lat]);
    buildCurrentMarker([gcj.lng, gcj.lat], '当前位置');
}

function drawFenceFromWGS84(lng, lat, radius) {
    const corrected = applyCorrectionWgs(lng, lat);
    const gcj = wgs84ToGcj02(corrected[0], corrected[1]);
    if (fenceCircle) {
        map.remove(fenceCircle);
    }
    buildCurrentMarker([gcj.lng, gcj.lat], '围栏中心');
    fenceCircle = new AMap.Circle({
        center: [gcj.lng, gcj.lat],
        radius: radius,
        strokeColor: '#3366FF',
        strokeWeight: 3,
        fillColor: '#3366FF',
        fillOpacity: 0.15
    });
    map.add(fenceCircle);
    map.setFitView([fenceCircle, currentMarker]);
}

function removeFence() {
    if (fenceCircle) {
        map.remove(fenceCircle);
        fenceCircle = null;
    }
}

function drawTrackFromWGS84(points) {
    const validPoints = (points || []).map(item => {
        const corrected = applyCorrectionWgs(item[0], item[1]);
        const gcj = wgs84ToGcj02(corrected[0], corrected[1]);
        return [gcj.lng, gcj.lat];
    });
    if (!validPoints.length) {
        return;
    }
    if (trackPolyline) {
        map.remove(trackPolyline);
    }
    trackPolyline = new AMap.Polyline({
        path: validPoints,
        strokeColor: '#00B578',
        strokeWeight: 6,
        lineJoin: 'round',
        lineCap: 'round'
    });
    map.add(trackPolyline);
    map.setFitView([trackPolyline]);
}

function clearTrack() {
    if (trackPolyline) {
        map.remove(trackPolyline);
        trackPolyline = null;
    }
}

function addPhotoMarkerFromWGS84(lng, lat, photoUri) {
    const corrected = applyCorrectionWgs(lng, lat);
    const pos = wgs84ToGcj02(corrected[0], corrected[1]);
    const marker = new AMap.Marker({

```

```

        position: [pos.lng, pos.lat],
        anchor: 'bottom-center',
        content:
            '<div style="position:relative;width:26px;height:34px;">' +
            '<div style="width:26px;height:26px;border-
radius:50%;background:#FF6A00;color:white;font-size:15px;display:flex;align-
items:center;justify-content:center;box-shadow:0 3px 8px rgba(0,0,0,0.25);"><img alt="camera icon" data-bbox="750 150 765 165"/></div>' +
            '</div>',
        title: '拍照定位点'
    });

    marker.on('click', function() {
        const infoWindow = new AMap.InfoWindow({
            content:
                '<div style="padding:8px;max-width:220px;word-break:break-all;">' +
                '<div style="font-weight:bold;margin-bottom:6px;">拍照定位点</div>' +
                '<div style="font-size:12px;">' + photoUri + '</div>' +
                '</div>',
            offset: new AMap.Pixel(0, -20)
        });
        infoWindow.open(map, [pos.lng, pos.lat]);
    });

    photoMarkers.push(marker);
    map.add(marker);
    map.setZoomAndCenter(17, [pos.lng, pos.lat]);
}

function toggleSatellite(enable) {
    if (enable) {
        if (!satelliteLayer) {
            satelliteLayer = new AMap.TileLayer.Satellite();
        }
        if (!roadNetLayer) {
            roadNetLayer = new AMap.TileLayer.RoadNet();
        }
        map.setLayers([satelliteLayer, roadNetLayer]);
    } else {
        map.setLayers([new AMap.TileLayer()]);
    }
}

function searchPlace(keyword) {
    AMap.plugin(['AMap.PlaceSearch'], function() {
        const placeSearch = new AMap.PlaceSearch({ pageSize: 1, pageIndex: 1 });
        placeSearch.search(keyword, function(status, result) {
            if (status === 'complete' && result && result.poiList &&
result.poiList.pois.length > 0) {
                const poi = result.poiList.pois[0];
                map.setZoomAndCenter(16, poi.location);
                buildCurrentMarker(poi.location, poi.name);
            } else {
                setStatus('未搜索到地点: ' + keyword);
            }
        });
    });
}

function clearRoutePanel() {
    const panel = document.getElementById('routePanel');
    if (panel) {
        panel.innerHTML = '';
        panel.style.display = 'none';
    }
}

function clearAllRouteServices() {
    if (driving) {
        driving.clear();
    }
    if (riding) {
        riding.clear();
    }
    if (walking) {
        walking.clear();
    }
}

```

```

        clearRoutePanel();
    }

function clearRoute() {
    clearAllRouteServices();
    setStatus('路线已清除');
}

function planRoute(startKeyword, endKeyword, mode) {
    const panel = document.getElementById('routePanel');
    if (panel) {
        panel.style.display = 'block';
        panel.innerHTML = '';
    }

    clearAllRouteServices();

    if (mode === 'driving') {
        AMap.plugin(['AMap.Driving'], function() {
            driving = new AMap.Driving({
                map: map,
                panel: 'routePanel',
                policy: AMap.DrivingPolicy.LEAST_TIME,
                showTraffic: true,
                autoFitView: true
            });

            driving.search(
                [{ keyword: startKeyword }, { keyword: endKeyword }],
                function(status) {
                    setStatus(status === 'complete' ? '驾车路线规划成功' : '驾车路线规划失败');
                }
            );
        });
        return;
    }

    if (mode === 'riding') {
        AMap.plugin(['AMap.Riding'], function() {
            riding = new AMap.Riding({
                map: map,
                panel: 'routePanel',
                autoFitView: true
            });

            riding.search(
                [{ keyword: startKeyword }, { keyword: endKeyword }],
                function(status) {
                    setStatus(status === 'complete' ? '骑行路线规划成功' : '骑行路线规划失败');
                }
            );
        });
        return;
    }

    AMap.plugin(['AMap.Walking'], function() {
        walking = new AMap.Walking({
            map: map,
            panel: 'routePanel',
            autoFitView: true
        });

        walking.search(
            [{ keyword: startKeyword }, { keyword: endKeyword }],
            function(status) {
                setStatus(status === 'complete' ? '步行路线规划成功' : '步行路线规划失败');
            }
        );
    });
}

function initAMap() {
    try {
        if (typeof AMap === 'undefined') {
            setStatus('AMap 未定义: 脚本未正确加载');
            return;
        }
    }
}

```

```

    }

    map = new AMap.Map('container', {
        zoom: 15,
        center: [116.397428, 39.90923],
        viewMode: '3D'
    });

    map.plugin(['AMap.ToolBar', 'AMap.Scale'], function () {
        map.addControl(new AMap.ToolBar());
        map.addControl(new AMap.Scale());
    });

    setStatus('高德地图加载成功');
} catch (e) {
    setStatus('地图初始化失败: ' + JSON.stringify(e));
}
}

window._AMapSecurityConfig = {
    securityJsCode: 'a6b3a21770895d71f19b1ba127328d9b'
};

window.pushGnssObservation = pushGnssObservation;
window.refreshGnssRealtimePanels = refreshGnssRealtimePanels;
window.showCurrentLocationFromWGS84 = showCurrentLocationFromWGS84;
window.drawFenceFromWGS84 = drawFenceFromWGS84;
window.removeFence = removeFence;
window.drawTrackFromWGS84 = drawTrackFromWGS84;
window.clearTrack = clearTrack;
window.addPhotoMarkerFromWGS84 = addPhotoMarkerFromWGS84;
window.toggleSatellite = toggleSatellite;
window.searchPlace = searchPlace;
window.planRoute = planRoute;
window.clearRoute = clearRoute;
window.openGnssTool = openGnssTool;
window.closeGnssTool = closeGnssTool;
window.fillLatestCoord = fillLatestCoord;
window.runCoordTransform = runCoordTransform;
window.parseNmeaQuality = parseNmeaQuality;
window.toggleFusionMode = toggleFusionMode;
window.showFusedPointOnMap = showFusedPointOnMap;
window.showCorrectedPointOnMap = showCorrectedPointOnMap;
window.enableCorsMode = enableCorsMode;
window.enablePppMode = enablePppMode;
window.disableCorrectionMode = disableCorrectionMode;
</script>

<script>

src="https://webapi.amap.com/maps?v=2.0&key=ff0dddeb675330a76de6022267b0d78b&plugin=AMap.Plac
ceSearch,AMap.AutoComplete,AMap.Driving,AMap.Riding,AMap.Walking"
    onload="initAMap()"
    onerror="loadAMapFailed()">
</script>
</body>
</html>

```


附录 E 关键函数与调用关系说明

本附录面向代码阅读与答辩说明，对 index.ets 与 amap.html 中的关键函数进行索引化整理。通过方法职责、输入参数、输出结果与调用关系的表格化描述，可以帮助读者快速理解系统的控制链路和数据流向。

E.1 index.ets 关键方法说明

方法名	所在位置	主要职责	调用关系
requestBasicPermissions	鸿蒙端	申请位置与相机权限	被 getCurrentPosition 调用
getCurrentPosition	鸿蒙端	获取当前位置并更新经纬度	被定位、围栏、轨迹、拍照、实时轮询调用
pushGnssObservationToWeb	鸿蒙端	把样本推送至网页端	被 getCurrentPosition 调用
pollLatestLocationOnce	鸿蒙端	执行一次实时轮询刷新	被 startRealtimeMode 周期调用
startRealtimeMode	鸿蒙端	开启实时动态模式	被 toggleRealtimeMode 调用
stopRealtimeMode	鸿蒙端	关闭实时动态模式	被 toggleRealtimeMode/aboutToDisappear 调用
locateCurrentOnMap	鸿蒙端	执行一次手动定位并在地图展示	被首页/工具页定位按钮调用
doSearch	鸿蒙端	搜索关键字转义并发起搜索	被首页搜索按钮调用
doNavigate	鸿蒙端	收集起终点与模式并发起路线规划	被导航页开始导航按钮调用
openGnssToolWithCurrent	鸿蒙端	先取最新样本再打开 GNSS 工具页	被精度评估、误差分析、欺骗检测等调用

从调用关系上看，index.ets 的控制逻辑具有明显的“中心控制器”特点：所有用户操作首先由鸿蒙端接收，然后通过 runMapJs 将参数转发到网页端。这种模式将系统能力调用与地图分析展示进行了清晰分工。

E.2 amap.html 关键函数说明

函数名	功能类别	主要职责	与实时模式关系
initAMap	地图初始化	创建地图实例并加载控件	实时样本展示的基础
searchPlace	POI 搜索	调用高德 PlaceSearch 检索地点	与实时模式无直接耦合
planRoute	路径规划	完成驾车/骑行/步行路线绘制	与实时模式并行存在
drawFenceFromWGS84	围栏绘制	绘制圆形电子围栏	可配合实时样本扩展预警
drawTrackFromWGS84	轨迹绘制	根据样本折线绘制轨迹	实时模式开启后持续重绘
pushGnssObservation	GNSS 样本缓存	保存新样本并执行改正/融合前处理	实时模式核心入口
refreshGnssRealtimePanels	页面刷新	根据当前工具页重绘结果面板	实时模式核心刷新逻辑
computeScatterStats	精度统计	计算均值、平均散布、RMS、CEP95	样本越多越稳定
computeSpoofAnalysis	异常检测	计算风险分数、风险等级与触发因素	实时模式下动态更新
computeFusionResult	组合导航	根据样本窗口计算融合点、速度、航向	实时模式下动态更新
computeCorrectionStats	改正模拟	计算改正前后 RMS、CEP95 与均值	实时模式下动态更新

网页端函数整体围绕 `gnssState` 展开。`gnssState` 既存储定位样本、质量指标和异常结果，也记录当前是否开启融合模式、改正模式以及当前活动工具页。正因为有这一统一状态对象，GNSS 页面才能在实时模式下实现持续刷新。

E.3 关键调用链分析

调用链一：实时定位刷新链。`startRealtimeMode` -> `pollLatestLocationOnce` -> `getCurrentPosition` -> `pushGnssObservationToWeb` -> `pushGnssObservation` -> `refreshGnssRealtimePanels`。该链路负责把系统定位能力转化为网页端连续样本流。

调用链二：GNSS 工具联动链。用户点击卡片 -> `openGnssToolWithCurrent/openGnssTool` -> `window.openGnssTool(tool)` -> 进入对应渲染分支 -> 调用统计或分析函数 -> 输出指标卡片。该链路负责把用户需求转化为工具界面。

调用链三：轨迹动态更新链。实时轮询获取样本 -> TrackPoint 数组累积 -> drawTrackFromWGS84 -> 网页端 Polyline 重绘。该链路体现了位置时序样本与空间图形之间的对应关系。

附录 F 主要功能界面图集与版位说明

为便于课程提交与汇报展示，本附录根据当前真机运行结果，集中整理实时动态 GNSS 增强版 APP 的核心功能界面图。各图片已按功能逻辑重新排序，并对应放置到最合适的版位说明中。

F.1 实时首页与状态总览界面



图 F-1 实时首页与状态总览界面

该界面位于首页分栏，集中展示搜索框、定位当前位置、开启实时模式、拍照定位和卫星图切换按钮，并以卡片形式给出经度、纬度、实时模式状态和照片状态，是整套实时动态模式的总入口。

F.2 GNSS 总览实时界面

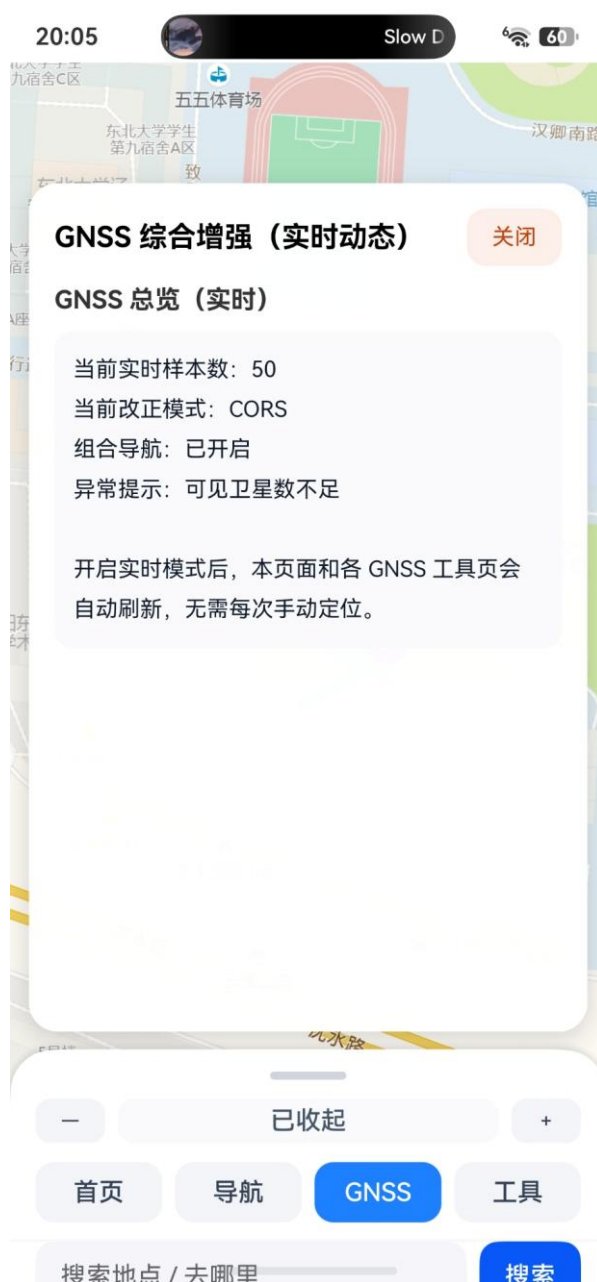


图 F-2 GNSS 总览实时界面

该界面用于汇总当前实时样本数量、改正模式、组合导航状态和异常提示，适合作为 GNSS 七项增强能力的综合监视入口。

F.3 实时坐标转换输入界面

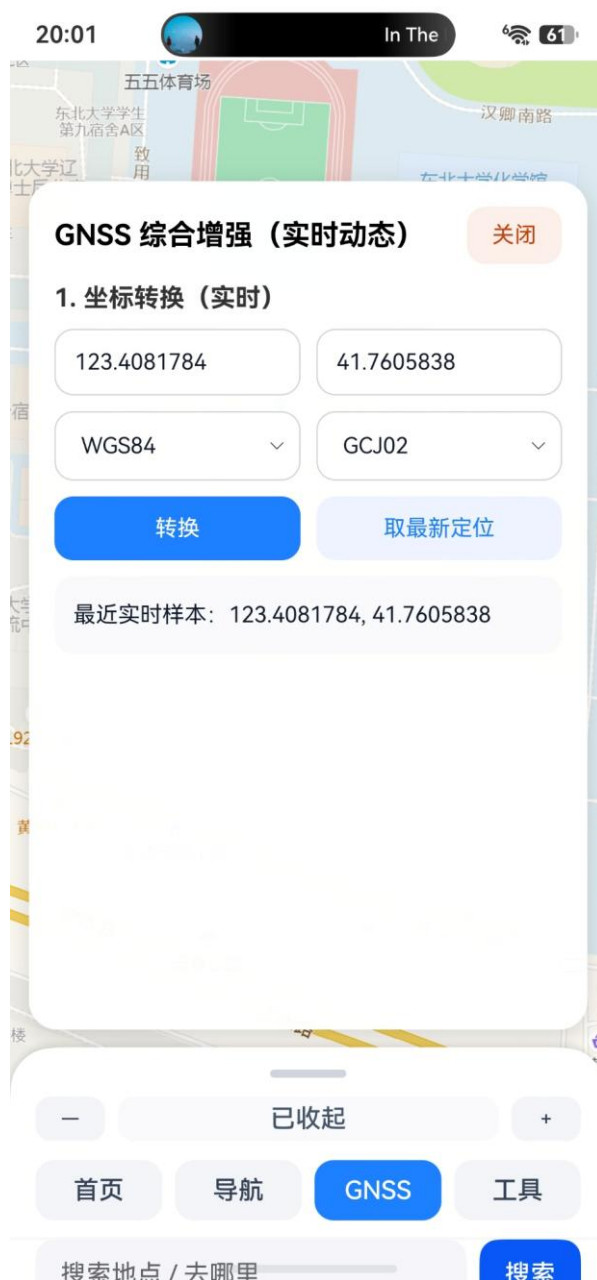


图 F-3 实时坐标转换输入界面

该界面展示坐标转换模块在实时模式下自动填充最近样本的能力，用户可以直接基于最新 WGS84 坐标执行坐标系转换，无需再次手动定位。

F.4 实时坐标转换结果界面

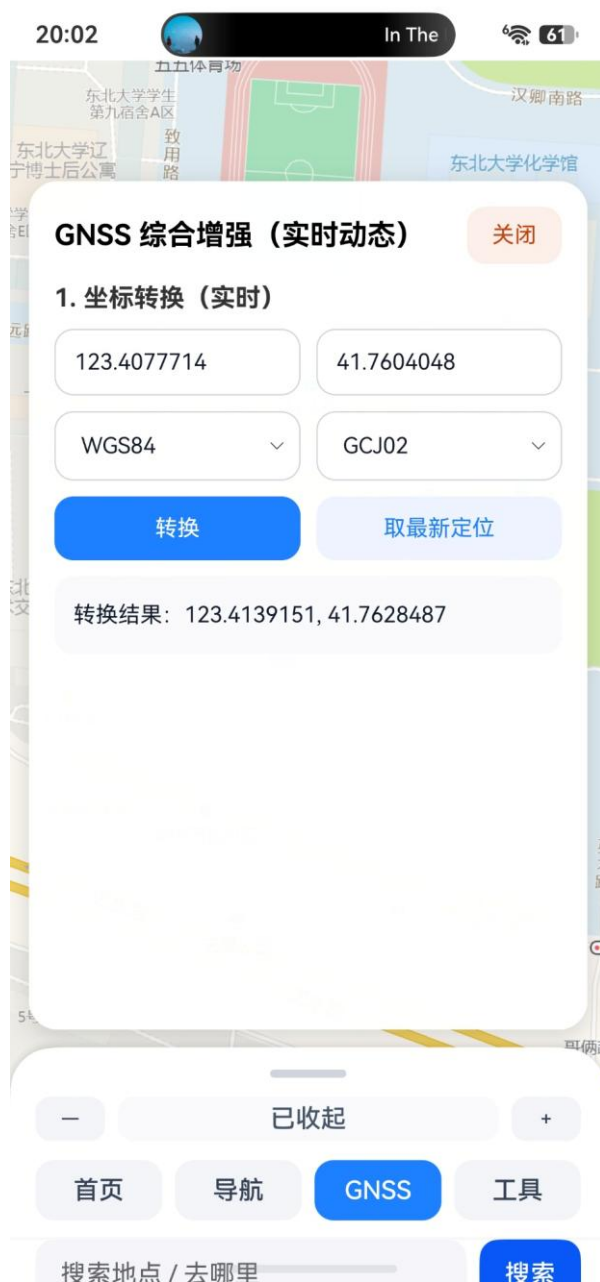


图 F-4 实时坐标转换结果界面

该界面展示坐标转换模块的结果输出状态。在 WGS84 到 GCJ02 的转换中，系统会将最新样本送入网页侧计算逻辑，并输出实时转换结果。

F.5 GNSS 实时精度评估界面



图 F-5 GNSS 实时精度评估界面

该界面展示实时样本下的精度统计结果，包括样本数、平均散布、RMS、CEP95 和均值坐标。页面会随着实时定位样本更新自动刷新。

F.6 GNSS 实时质量评估界面



图 F-6 GNSS 实时质量评估界面

该界面用于粘贴与解析 GSA/GSV NMEA 语句，计算可见卫星数、平均 C/N0、PDOP、HDOP 与 VDOP 等定位质量指标。

F.7 GNSS 实时误差源分析界面



图 F-7 GNSS 实时误差源分析界面

该界面根据实时样本自动计算散布 RMS、平均速度、航向变化和样本数，并给出多路径误差、卫星几何误差、遮挡衰减与接收机噪声等误差源解释。

F.8 GNSS 组合导航待开启界面



图 F-8 GNSS 组合导航待开启界面

该界面展示组合导航模块在开启前的状态。此时原始最新点与融合平滑点一致，速度、航向角和平滑 RMS 均接近静止状态。

F.9 GNSS 组合导航运行界面



图 F-9 GNSS 组合导航运行界面

该界面展示组合导航模块开启后的实时结果。系统基于时间窗平滑更新融合点坐标，并同步刷新平滑窗口、瞬时速度、航向角与平滑 RMS。

F.10 融合点地图显示效果界面



图 F-10 融合点地图显示效果界面

该界面展示组合导航结果映射回地图后的可视化效果。蓝色圆点表示当前位置，紫色 F 标记表示融合平滑点，绿色折线反映实时轨迹与方向变化。

F.11 GNSS 欺骗与异常检测界面



图 F-11 GNSS 欺骗与异常检测界面

该界面用于根据实时样本计算风险等级、风险分数、最大突跳、最高速度、突跳次数与航向突变，并基于触发因素给出诊断结论。

F.12 CORS 模拟改正实时界面



图 F-12 CORS 模拟改正实时界面

该界面展示 CORS 模拟改正后的效果。系统给出改正前 RMS、改正后 RMS、RMS 改善比例、改正后 CEP95 以及改正前后均值坐标。

F.13 PPP 平滑改正实时界面



图 F-13 PPP 平滑改正实时界面

该界面展示 PPP 平滑模式下的改正结果。相较于 CORS 模拟偏移改正，PPP 强调通过时间窗平滑模拟精密单点改进。